

엔트리로 파이썬 기초 입문하기

본 서는 엔트리의 블록기반 코딩으로 코딩(프로그래밍)을 익힌 이후 파이썬 언어를 사용하는 텍스트기반 코딩으로 넘어가고자 하는 분들에게 중간 다리 역할을 하는 "엔트리-파이썬" 코딩을 통해 교육목적의 블록 코딩 개발에서 실전개발을 위한 텍스트 코딩으로의 용이한 전환을 꾀하고자 하는 분/블록 코딩을 교육한 이후 학습자의 텍스트 코딩으로의 원활한 전환을 돕고자하는 교육자들을 위해 씌어졌습니다.

변경내역:

버전	날짜	내역
v1	2023/11/06	첫 버전 공개
v1.1	2024/02/25	함수 파트 일부 수정
v1.2	2024/07/08	순차/병렬처리 일부 수정

저자: JJ (comseong@gmail.com)

참고 자료1: 엔트리 > [공식교육자료](#) > 엔트리 파이썬

참고 자료2: 엔트리 오프라인에 내장된 "엔트리 파이썬 이용안내 ppt(엔트리파이썬 메뉴얼, 엔트리파이썬 기본예제)"

저작권: <https://creativecommons.org/licenses/by-nc-sa/4.0/>



여러분의 후원은 콘텐츠 제작에 큰 힘이 됩니다!!

토스로 커피 한 잔(3000원) 후원하기



여러분의 후원은 콘텐츠 제작에 큰 힘이 됩니다!!

pay



3,000원

JJ

커피 한 잔 후원하기

Table of contents

- [엔트리로 파이썬 기초 입문하기](#)
- [1. 엔트리 vs 엔트리-파이썬](#)
- [2. 엔트리-파이썬의 시작](#)
- [3. 엔트리-파이썬으로 파이썬 언어의 기본문법을 익히자](#)
 - [3.1 Hello World 예제코드의 이해](#)
 - [3.2 입/출력 \(Input/Output\)](#)
 - [3.3 변수 \(Variable\)](#)
 - [3.4 조건 \(Condition\)](#)
 - [3.5 반복 \(Loop\)](#)
 - [3.6 리스트 \(List\)](#)
 - [3.7 무작위 수 \(Random\)](#)
 - [3.8 함수 \(Function\)](#)
- [4. 엔트리-파이썬으로 프로그래밍의 핵심 개념을 익히자](#)
 - [4.1 순차/병렬처리 \(Serial/Parallel\)](#)
 - [4.2 절차 지향형 \(Procedural\)](#)
 - [4.3 이벤트 주도형 \(Event-Driven\)](#)
 - [4.4 객체 지향형 \(Object-Oriented\)](#)
- [5. 부록](#)
 - [엔트리-파이썬 단축키 모음](#)

1. 엔트리 vs 엔트리-파이썬

엔트리는 네이버 자회사 비영리 기관인 **커넥트 재단**에서 제작하고 후원하는 오픈소스 교육용 코딩언어(EPL: Educational Programming Language)이다. 특별히 한국에서는 코딩교육에 가장 많이 채용되고 활동되는 블록기반 코딩언어(a high-level block-based visual programming language, 이하 블록코딩 언어)이다.

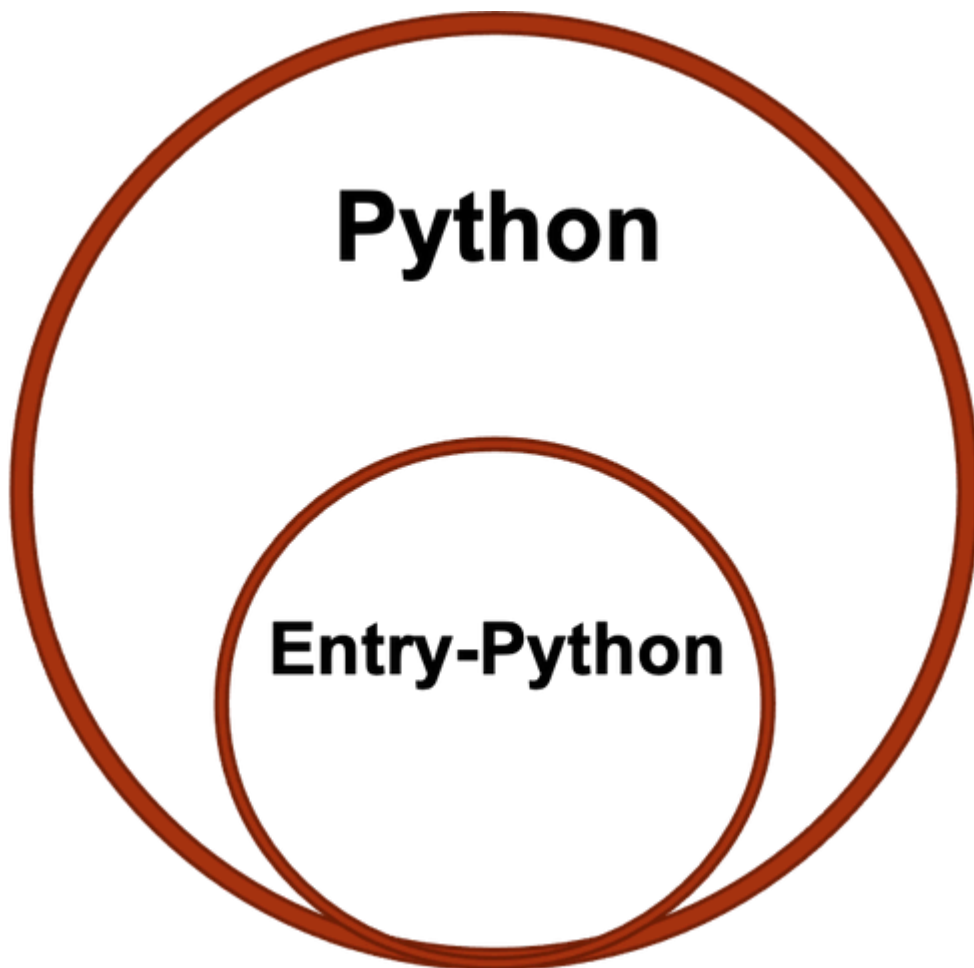
기존에 전 세계적으로 가장 잘 알려진 MIT의 **스크래치**에 비해 큰 강점이기도 한 차별점이 있는데, 기획 당시부터 블록 기반 코딩으로 입문하여 궁극적으로 텍스트 기반 코딩(text-based programming language, 이하 텍스트코딩 언어) 입문까지를 커버하는 언어로 만들어졌기 때문이다. 엔트리 안에서 텍스트코딩 입문용 언어로 선택된 것은 파이썬(Python)이다. 파이썬 언어는 텍스트코딩 언어 중에 가장 배우기 쉬우면서 최근에 각광을 받는 특별히 데이터 분석과 인공지능 분야에서 활용성이 뛰어난 언어이기 때문에 텍스트코딩 언어기반 코딩교육에 첫 입문용 언어로 보편적으로 활용되고 있다.

[!NOTE]

파이썬(Python): 파이썬은 1991년 네덜란드계 소프트웨어 엔지니어인 귀도 반 로섬이 발표한 고급 프로그래밍 언어로, '인터프리터'를 사용하는 **객체지향** 언어이자 플랫폼에 독립적인, **동적 타이핑**(dynamically typed) 대화형 언어이다. 파이썬의 강력한 라이브러리와 풍부한 생태계를 통해, 데이터를 수집하고 분석하며 시각화할 수 있다. 데이터 분석 분야에서 파이썬의 사용이 널리 퍼진 이유 중 하나는, 다른 프로그래밍 언어에 비해 비교적 쉽고 간편하게 사용할 수 있기 때문이다.

출처: 위키피디아

그러나, 엔트리에 탑재된 "**엔트리-파이썬**"은 **엔트리 전용 파이썬 언어**라고 할 수 있다. 오리지널 파이썬 언어의 기본의 문법정도를 익히기 위한 교육목적의 **오리지널 파이썬의 미니 파이썬 버전**정도로 간주할 수 있고, 따라서, 완전한 파이썬 그 자체는 아니기 때문에 오리지널 파이썬 문법 중 지원하지 못하는 문법이 상당히 있다는 것을 항상 염두하면서 코딩을 하자.



우리의 목표는 "엔트리-파이썬"에서 최대한 빠르게 파이썬의 기본 문법을 익히고, 하루 빨리 **오리지널 파이썬**으로 전향해 블록코딩을 완전히 떠난 이제는 텍스트코딩으로만 코딩(프로그래밍)하는 것이 궁극적인 목표이기 때문에 엔트리-파이썬 안에서 너무 큰 기대와 목적을 초과하는 지나친(?)

사용은 자제하는 것이 정신건강에 좋을 것 같다. 그 이유는 안타깝게도 엔트리-파이썬 자체는 생각보다 불안정성이 여전하여(예를들어 코드 상에 문법오류가 전혀 없음에도 오류가 있다고 에러를 내며 실행이 안되는 등등, 겪어보면 안다;) 다소 복잡한 코딩을 시작하게 되면 그러한 문제에 부딪힐 가능성이 점점더 증가하기 때문이다.

[!NOTE]

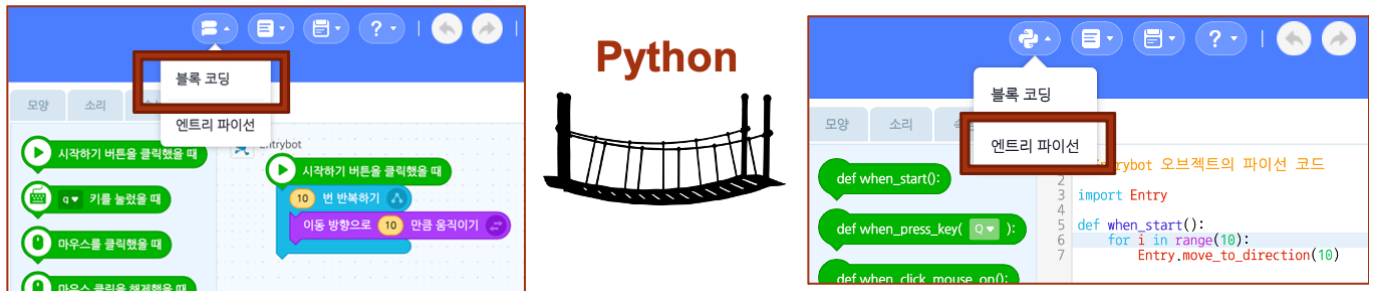
따라서, 위에서 언급된 오리지널 엔트리 버전에서 엔트리-파이썬 불안정성 때문에 코딩하면서 겪는 스트레스를 최대한 제거한 저자의 [오프라인 커스텀 엔트리 버전](#)을 다운로드 하여 설치 후 이 책의 예제들의 실습 및 수업에 활용할 것을 강력하게 추천한다.

다시한번 요약하면, 엔트리-파이썬은 블록코딩에서 첫 텍스트코딩의 넘어가는 분들이 대상이고, 생짜배기로 텍스트코딩 세계에 막 들어갔을 때의 막막함(?)의 장벽을 낮춰주기 위함이며, 우리에게 이미 익숙한 블록코딩 환경 안에서 텍스트로 일일이 키보드 자판을 타이핑하며 텍스트코딩 해본 것이 블록코딩으로 했을 때와 큰 차이 없이 동작하는 것을 경험하며 텍스트코딩이 그렇게 어려운 것이 아니구나 라는 선입견을 없애는 정도만 해도 엔트리-파이썬이 목적인 그 소기의 목적달성이라고 할 수 있겠다.

2. 엔트리-파이썬의 시작

엔트리 (블록 코딩) <-> 파이썬 (텍스트 코딩) 변환

이미 엔트리 사용자라면 엔트리에 대한 사전 지식을 통해서든 엔트리 작동법을 탐구 중 우연히 해당 기능을 발견했을 수도 있을 것이다. 엔트리 상단 메뉴바에서 코드변환 아이콘을 눌러 두 코드 간의 전환을 시도 하면 된다.



이처럼 엔트리는 나의 블록코딩의 언어가 텍스트 코딩언어(파이썬)로는 어떻게 되는지를 코드 변환을 통해 확인해 볼 수 있을 뿐만 아니라, 엔트리-파이썬 코딩 에디터 창을 통해 직접 텍스트 코딩을 하고 이를 실행해 볼 수 있는 기능까지도 제공한다. 이러한 "엔트리-파이썬"은 학습자나 교사가 엔트리에서 "블록 코딩에서 시작해 텍스트 코딩까지 익히기" 라는 목표를 이룰 수 있도록 그 중간을 매개하는 연결다리와 같은 역할이라 할 수 있을 것이다.

블록코딩에서 첫 텍스트코딩으로 넘어오면서 겪는 경험

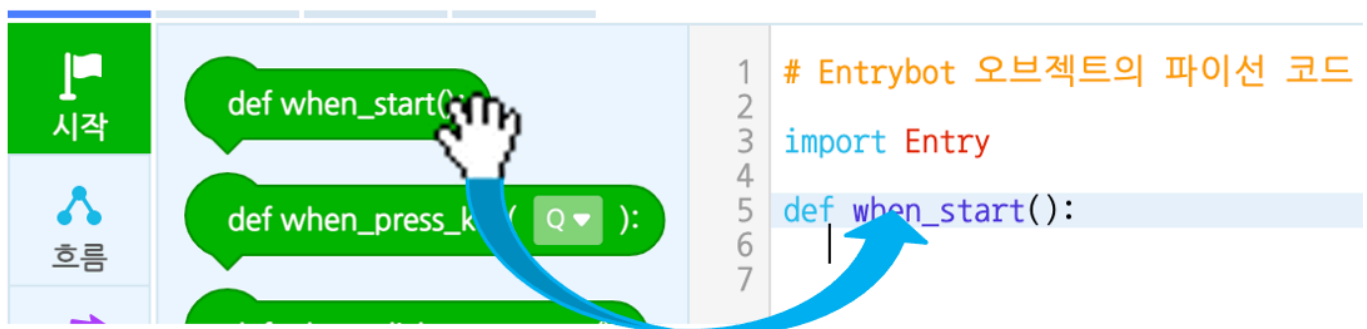
일반적으로 블록코딩에서 첫 텍스트코딩으로 넘어오면서 겪는 큰 당혹감은 기존에 블록코딩에서처럼 단순히 블록 카테고리에 위치한 블록들을 코딩에디터(코드조립소) 영역으로 옮겨 놓으면서(Drag & Drop) 조립하면서 의미있는 블록더미를 만드는 이해하기 쉽고 직관적이고 편리한 방식으로 코딩할 수는 없고, 대신에 마치 긴 글을 작문하듯이 글자 하나하나를 일일이 수많은 텍스트 타이핑을 하면서 코딩해 나가야 한다는 것이다.

이때 우리는 이러한 코딩과정에서 기존 블록코딩에서는 경험해 보지 못한 에러(error) 상황들을 마주치게 되며, 그 사소해 보이는 에러(또는 오류)가 내가 코딩한 프로그램 전체를 아예 실행할 수 없게 만든 것은 다소 당혹스럽게 느껴지기 까지 하다. 왜냐면 기존 블록코딩에서는 코딩을 잘못된 경우, 프로그램이 오동작 하는 정도이지 아예 전체가 실행이 불가하지는 않았기 때문이다.

에러는 타이핑한 텍스트가 오타인 경우엔 이해가 되나 오타라기 보다 겨우 알파벳 대소문자 정도의 차이, 점 하나를 찍고 안찍고의 차이임에도 불구하고, 프로그램을 실행하려고 하면 그것을 에러라고 내뱉으며 프로그램 자체를 아예 실행할 수 없게 된다. 텍스트 코딩은 그런 면에서 매우 주의깊고 정확성 있는 코딩이 필요하고, 그 외에도 내가 코딩하는 내용 자체가 해당 코딩언어가 요구하는 문법 체계와도 맞는지까지 한번 더 주의를 살펴 코딩해야 한다. 이러한 면 때문에 처음에 겪는 당혹감은 누구에게나 당연한 것이고, 다만, 이러한 코딩방식에 적응되고 익숙해지기까지는 모두에게 어느 정도의 시간이 필요한게 사실이다.

엔트리-파이썬에서 드래그앤드롭(Drag&Drop) 방식으로 코딩하기

엔트리-파이썬에서는 위에서 언급한 텍스트 코딩시 겪게 되는 타이핑 오류로 인한 실수를 막을 수 있는 방법의 아이디어로 기존 블록코딩에서의 코딩방법을 차용해 그림에서처럼 드래그앤드롭 방식으로 코딩을 시도해 볼 수 있다.



이러한 방법이 단순 타이핑 실수로 인한 에러를 줄여주는 효과와 키보드 타이핑하는 시간자체를 줄여주는데는 편리함이 있지만, 그렇다 할지라도 위에서 언급한데로 텍스트코딩시 사전에 필수로 알아야 하는 해당 코딩언어의 문법체계는 사전에 잘 알고 있다는 전제를 가졌을 때에만 활용할 수 있는 코딩방법이라고 할 수 있다. 그러므로 이제 우리가 우선적으로 해야할 일을 파이썬 언어의 기본 문법체계를 익히는 일부터 시작해야 할 것이고, 다음 정부터 이를 하나씩하나씩 차근차근 배워보도록 하겠다.

3. 엔트리-파이썬으로 파이썬 언어의 기본문법을 익히자

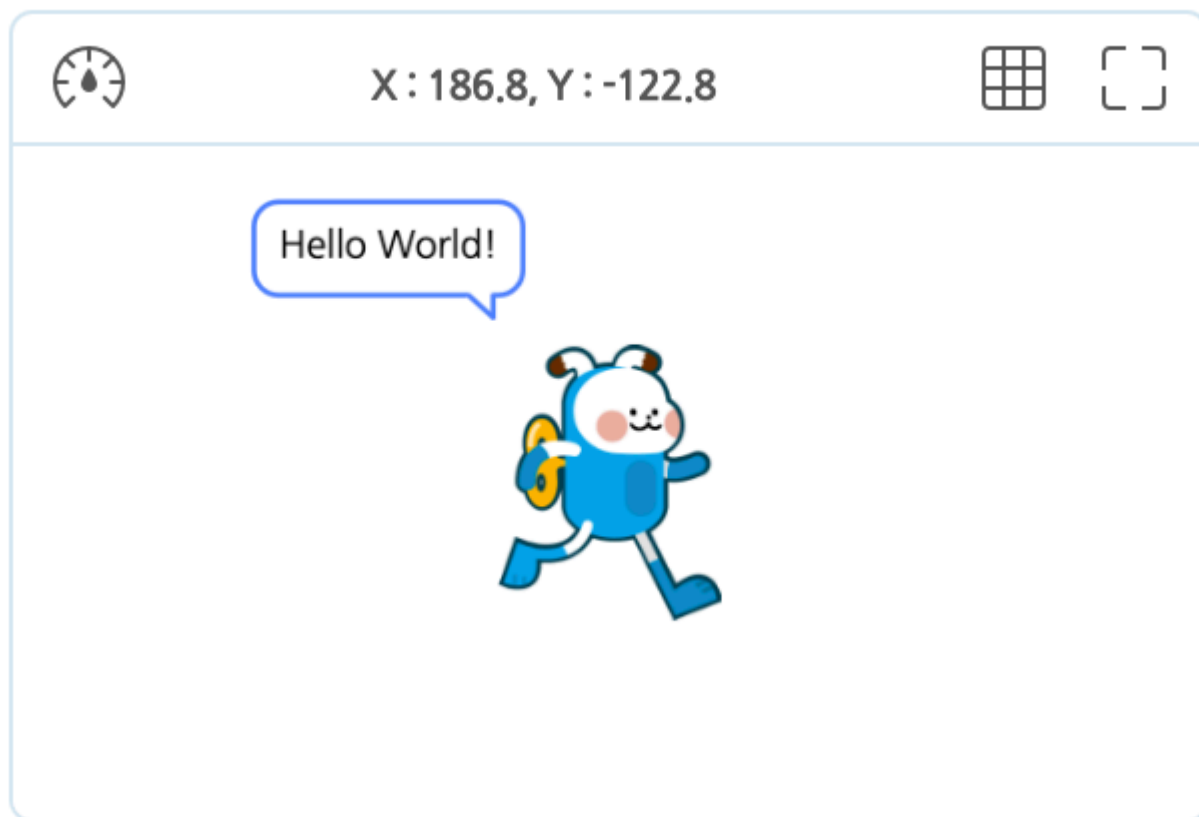
텍스트 코딩의 문법체계를 배운다는 것은 마치 우리가 다른 나라 외국인과 대화하기 위해 외국어를 공부하는 과정과 유사하다. 다만, 대상이 외국사람이 아닌 컴퓨터라는 것이다. 우리가 처음 외국어를 배울때 처럼 해당 언어를 표기하는 문자를 아는 것 부터 시작해(다행히 컴퓨터와의 대화는 우리가 이미 알고 있는 영어를 기반하는 라틴 알파벳으로만 표기), 문자들의 조합으로 음절(syllable)이 되고, 음절의 조합으로 단어가 되고, 그 단어들이 문장을 구성하는 문법체계(예를들어 주어+동사+목적어 등의 순서)에 맞추어 구성될 때에야 비로소 상호 간에 대화가 가능하다.

우리는 이미 어떤 외국어 배우는 것을 시도해 본 경험들(최소한 영어 등)이 있고, 이 부분이 우리에게겐 장점이면서 동시에 언어 배우는 것은 어렵다는 선입견을 갖을 수 있는데 컴퓨터 코딩언어는 최소한 인간이 사용하는 언어가 갖는 복잡성 보다는 훨씬 쉽고, 그 중에 파이썬이란 언어는 컴퓨터 코딩언어 중에 아주 쉬운 축에 속하기 때문에(특히나 기존에 영어에 대한 지식이 많으면 많을 수록) 첨부터 너무 부담을 갖지 않기를 바란다.

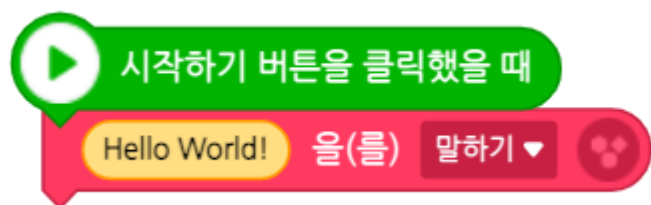
3.1 Hello World 예제코드의 이해

아마 다른 종류에 컴퓨터 코딩언어 책을 본 경험이 있다면, "Hello World!" 라는 한 문장을 화면에 출력하는 아주 간단한 내용을 코딩한 예제코드의 분석에서 출발하는 것을 본적이 있을 것이다. 우리도 역시 전형적인 이 예제 코드로부터 시작해 문법 익히기를 시작해보자.

실행결과



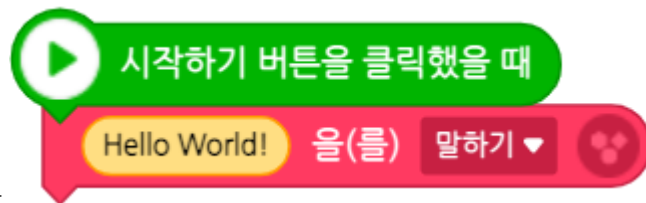
블록코딩



엔트리-파이썬

```
1 | # 엔트리봇 오브젝트의 파이썬 코드
2 |
3 | import Entry
4 |
5 | def when_start():
6 |     Entry.print("Hello World!")
```

앞으로 코드분석은 위와 같이 3개의 탭 형태로 구성하되 맨 처음 탭은 해당 코드의 실행결과를 나타내고, 중간과 마지막 탭은 엔트리-파이썬 / 블록 코딩 형태의 코드로 동일한 실행결과를 얻기 위한 각각의 코드를 비교해 볼 수 있도록 구성하였다.



이 예제의 경우 블록코딩으로는 단 2개의 블록만으로 코딩되는 아주 단순한 프로그램이다. 하지만, 이를 엔트리-파이썬으로 코딩했을 때는 무려(?) 6라인(줄)으로 구성된 코드(코드 가독성을 높이기 위해 넣은 중간마다 넣은 공백라인 포함)로 길어지게 된다. 당장은 여러분에게 이 예제를 직접 타이핑 해볼 것을 권하지 않겠다. 코딩은 이후에 얼마든지 할 기회가 있고,まずは 텍스트 코딩예제로서 가장 쉬운 코드라 불리는 이 6라인의 의미를 명확히 이해하는 것이 더 중요하다. 따라서, 여러분은 지금 당장은 이 내용을 블록코딩으로 코딩하고 이후 메뉴에 자동 코드변환하기 메뉴를 선택해 엔트리-파이썬 코드로 변환시켜보기 바란다.

1번 라인의 "# Entrybot 오브젝트의 파이썬 코드"의 의미는 무엇인가? 텍스트 코딩에서는 기호 하나하나 심지어 때론 공백까지도 어떤 의미와 목적을 갖고 있다는 것을 알고 있을 것이다. 따라서, 1번 라인에 처음 등장하는 #의 기호 역시 의미와 목적이 있을 것이라는 것을 유추할 수 있고, 현재 # 기호 뒤에 적혀진 내용을 통해 미루어 짐작해 볼 수 있는데, "이 기호로 시작해 이후로(단, 한 라인 이내) 적는 내용들을 주석(comment)이라고 간주하라"라는 의미가 된다.

[!NOTE]

텍스트 코딩에서 주석(comment)이라는 것의 의미를 이해해보자. 컴퓨터는 우리가 작성한 코드를 최종 실행을 위해 결국에는 실행과정에서 하나하나 해독해 나가게 되는데, 주석이란 부분을 만나면 그 부분은 실행과는 전혀 상관없는 것이니 해독없이 그냥 무시하고 지나가라는 우리 편에서의 의미전달로 받아드리고, 컴퓨터는 그 부분을 해독없이 건너 뛰어(skip) 지나간다.

그렇다면 주석은 왜 필요할까? 목적은 크게 2가지인데, 하나는 주석이란 그 말 자체가 갖는 의미대로 우리가 작성한 코드를 설명하는 즉, 추가적인 정보를 담아서 나중에 저자 본인이나 우리 동료들이 이 코드를 보았을 때, 빠르고 올바르게 이해할 수 있게 돕는 목적의 보충설명을 다는 용도이다. 두 번째는 코드의 논리적 오류(문법상의 오류는 없으나, 실행시 의도한대로 수행하지 못하는 오류)를 디버깅(Debugging, 오류를 찾아고치는 과정)하는 과정에서, 임시로 내 코드의 일부를 주석처리 함으로써 실행과정에서 임시로 내 코드의 일부를 수행없이 건너뛰어(skip) 지나가게 하면서 오류를 빨리 찾아내기 위한 목적에서 사용한다.

그 다음 **2번 라인**의 분석으로 넘어가자. 그냥 공백으로 비워둔 라인이다. 이러한 라인은 2, 4라인으로 총 2라인이 있다. 왜 공백 한 라인을 넣었는가? 이유는 간단하다. 컴퓨터가 코드해독시 의미를 주기 위한 목적은 전혀 없고, 인간인 우리편에서 코드를 읽을 때 한눈에 잘 들어와 읽히게 하려는 가독성을 주려는 목적에서다. 그런데 가독성은 생각보다 매우 중요한데, 우리는 이제 막 텍스트코딩에 입문한 입문자라 잘 와닿지 않겠지만, 조금만 규모가 있는 프로그램 개발로 넘어가면 나 혼자 개발하는게 아니라, 다른 개발자들과 협업하는게 일반적이다. 그래서, 다른 사람이 나의 코드를 쉽고 빠르고 편하게 읽을 수 있게 늘 배려하는 습관을 몸에 베게 하는게 필요하다. 이는 개발자들에게 빠져서는 안 될 필수덕목이라 할 수 있겠다. 그래서, 코딩을 하면서 적절하게 공백라인을 사용하고, 또 적절하게 주석을 사용하는 일은 매우 중요하다.

이제 **3번 라인**으로 넘어가자. **import Entry** 라는 두 단어를 맞았는데, 이 문장에서 import 키워드의 의미는 라이브러리(Library, 파이썬에서는 패키지(package), 그보다 더 작은 단위 모듈(Module)이라고도 불림)를 우리가 현재 코딩하는 파일의 외부에서 가져다가(import해서) 코딩에 활용해 쓰려고 한다라는 것이고, 그 위에 오는 Entry라는 단어는 우리가 필요한 라이브러리의 이름인데, 전체적으로 이 코드의 의미는 Entry(라이브러리의 이름은 그 라이브러리의 저자가 임의로 부여하게 됨)라는 라이브러리를 외부로부터 호출해 가져와달라는 의미이다. 그렇다면 이제 라이브러리가 무엇인지 알아볼 차례이다.

라이브러리란

[!NOTE]

라이브러리(Library): 소프트웨어를 개발할 때 컴퓨터 프로그램이 사용하는 **비휘발성 자원**의 모음이다. 여기에는 구성 데이터, 문서, 도움말 자료, 메시지 틀, **미리 작성된 코드**, **서브루틴**(함수), **클래스**, **값**, **자료형** 사양을 포함할 수 있다.

출처: 위키피디아

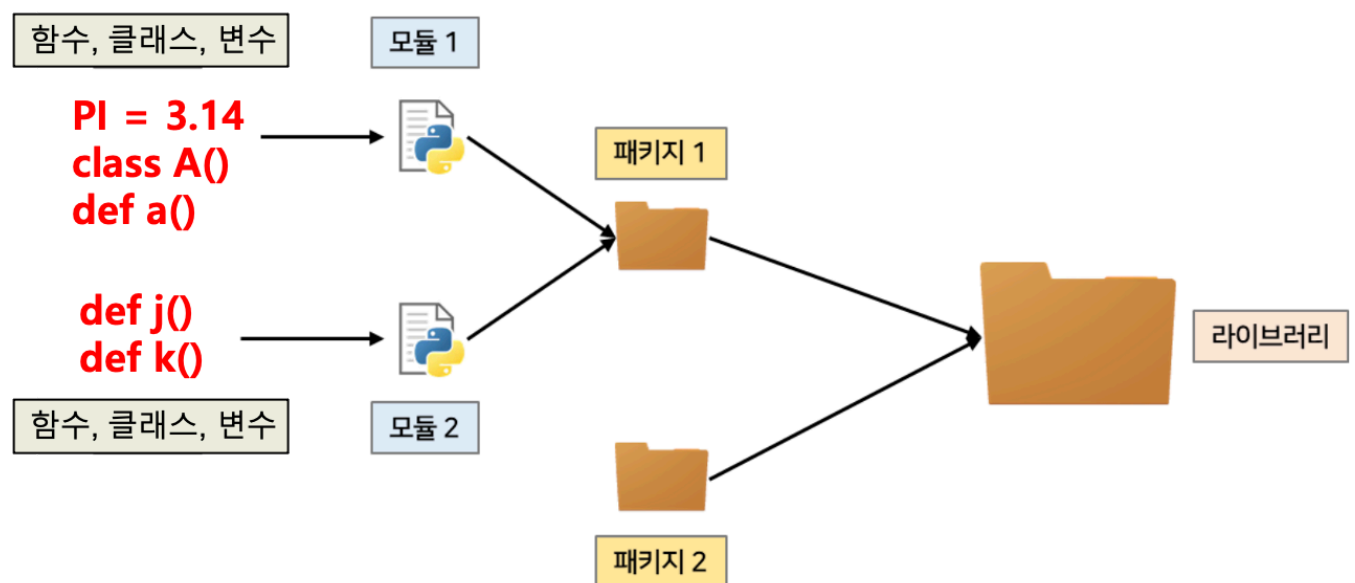
위에 라이브러리의 정의를 읽고 이해가 되었는가? 라이브러리를 간단히 설명하면 개발자들이 프로그램 개발시 빠르고 편리하게 개발하기 위한 목적으로 미리 작성해 놓은 코드덩어리들(주로 함수, 클래스, 변수 등)의 모음이라고 얘기할 수 있다. 당신은 이미 엔트리에서 함수(Function)를 사용해 봤고, 함수의 존재 목적에 대해 잘 이해하고 있는가? 함수의 주 목적이 무엇이었는가? 여러 곳에서 반복해서 사용하게 되는 코드덩어리를 함수

로 만들어 놓게 되면, 이후에는 그 함수블록 1개의 호출만으로 우리가 원하는 기능을 손쉽게 구현이 가능하여 코딩자체가 매우 간소화 되어 코딩의 효율이 증가하는 효과와 동시에 함수로 인해 간소화된 코드는 코드전체의 이해도 크게 증가하는 것을 경험했을 것이다.

그 밖에 함수의 더 유용하고 본질적인 목적은 한번 잘 만들어둔 함수를 프로그램 전체 코드 내 여러 곳에서(한 개의 오브젝트 또는 다량의 오브젝트 안에서) 사용함으로써(이를 코드의 재사용성이라 함) 한번 만들어진 프로그램이 이후의 변경(프로그램은 마치 살아있는 개체로서 한번 만들어지고 끝이 아닌 계속 반복적인 수정을 통해 발전진화해 나가는 과정을 겪음)에 대해 손쉽게 대응하려는 목적이 더 크다. 왜냐면 특정 기능을 함수로 만들고, 그 특정기능의 개선을 위해 함수자체의 내용을 수정하게 되면, 그 함수를 사용해 코딩한 모든 코드에서 한꺼번에 동일한 반영이 적용되기 때문이다.

혹시 지금 위해서 언급된 함수의 목적과 의미를 잘 이해하지 못하였다면, 귀하는 엔트리 블록코딩에서 함수를 사용해 코딩해 본 적이 없거나 함수의 존재목적을 잘 이해하지 못한채 사용했을 가능성이 크다. 여기서 함수개념 자체를 다시 재설명하는 것은 본 서의 범위를 넘어서는 것으로, 참고할 수 있는 [동영상 링크](#)를 남기니 필요한 분은 개인적으로 독학하는 것으로 같음하겠다.

잘 만든 함수 하나가 얼마나 유용한지 경험했다면, 이제 이러한 고민이 시작될 수 있다. 그 함수를 나 혼자만 쓰긴 너무 아깝지 않은가? 모든 개발자들이 작게는 우리 동료들이, 아니면 전세계의 모든 개발자가 다같이 함께 쓰면 쓸수록 더 좋은게 아닌가. 그래서, 그 잘 만들어진 유용한 함수들을 모두가 함께 쓰는 구조를 만들기 시작했는데 그게 라이브러리(또는 패키지)라고 불린다. 개발자들은 특히나 늘 이런 것에 관심이 많다. 왜냐면 그들은 기본적으로 다 극도의 효율성 추구자들이기 때문이다^^; 먼저, 개별 함수들을 한 곳에 모으고, 다시 함수들을 모은 그것을 또 한 군데 모아나가면서 구조를 만든다. 이를 그림으로 표현해 보면 다음과 같다.



이해하기 쉬운 적절한 비유로 무엇이 있을까 고민했을 때, 라이브러리(library)는 말을 붙인 의도가 있을 것 같은데, 말 그대로 도서관이고 도서관은 책들로 가득한데, 개별적인 단행본 개념의 단권짜리 책도 있지만, 때로는 시리즈물 전집책도 상당히 있는데, 그런 전집류를 패키지(package)라고 생각해 볼 수 있고, 단 권짜리 책들은 모듈(module), 마지막으로 책 안에는 책의 내용을 구성하면서 의미단위를 만드는 장, 절, 단락, 문장 등이 있는데 이것들을 클래스, 함수, 변수 등으로 비교해 설명해도 무리가 없지 않을까 한다.

지금은 그냥 라이브러리 안에는 우릴 위해 미리 남들이 개발해 놓은 유용한 코드문치(특별히 주로 함수들)로 가득하고, 우리는 그냥 그것을 가져다가 활용해 코딩을 한다 정도로만 알고 있어도 충분할 것 같다.

34 드디어 5번 라인에 도달했다. 이게 코드분석의 마지막이라고 생각해도 좋다. 왜냐면 5~6번 라인은 별개로 분리된 코드가 아닌 하나의 의미를 갖는 코드문치이기 때문이다.

```
5 | def when_start():
6 |     Entry.print("Hello World!")
```

def when_start(): 이 코드의 의미는 무엇일까? 앞에서 이미 수차례 언급된 바로 그 함수라는 것을 직접 만들고 있는 코드이다. 초장부터 함수를 이용하기 위해 외부에서 부르는 방법을 익히더니 이젠 직접 만드는 방법까지! 그만큼 텍스트 코딩에서는 함수가 필수적인 개념이다. 우선 파이썬 텍

스트코딩에서 일반적인 함수 만드는 문법을 알아보자.

함수 만드는(정의하는) 법

Function

□ Syntax

- Definition

```
def function_name (parameters):  
    statements
```

4 spaces or 1 tab

- Usage

```
function_name(argument)
```

위에 표준문법의 각 단어와 기호의 의미를 이해해 보자. 함수를 만들려면 문두 시작에 항상 *def*(definition의 약자)라는 키워드를 달고 시작한다. 이제부터 내가 적는 내용은 함수를 만드는(정의(definition)하는 거야라고 의미표현이다. 그리고, 이어서 내가 만들 함수의 이름을 의미있게 임의로 적는다. 그리고, 그 함수 호출시 호출자로부터 넘겨받을 값들(이를 프로그래밍 세계에서 파라미터라고 부름)을 괄호 안에 쉼표(,) 구분자를 사용해 나열한다. 그리고, 마지막에 콜론(:)를 기호를 입력함으로써 여기까지 함수뼈대의 끝이고, 이제 다음 라인부터는 함수 안에서 실제 수행해야할 코드내용이 채워질꺼라는 의미가 부여된다.

[!NOTE]

파라미터(parameter)와 아규먼트(argument) 차이

파라미터(parameter)

- 파라미터는 함수를 정의할 때 사용되는 변수입니다.
- 함수 정의에서 괄호 안에 나열되는 변수들을 의미합니다.

아규먼트 (Argument)

- 아규먼트는 함수를 호출할 때 실제로 전달되는 값입니다.
- 함수 호출 시 괄호 안에 전달되는 실제 값들을 의미합니다.

def when_start(): 를 위에 표준문법에 다시한번 빗대어 설명해 보면 *when_start* 라는 이름을 갖는 함수를 만드는데, 그 함수에 전달되어질 값(파라미터)은 전혀 없는(그래서, 괄호 안은 비워져있다) 함수를 만들겠다 라는 것이다.

34 이제 마지막 **6번 라인**으로 넘어가면 이 함수가 불러졌을 때 수행해야 할 내용을 적는데, *특이한 것은 다른 라인들과 달리 문두 시작전에 앞에 어느 정도의 공백을 띄우고(Tab 키를 1회 눌러 한번에 공백을 만들거나 또는 스페이스바를 4회 눌러 4개의 공백을 넣음) 코드를 적기 시작한다는 것이다.* 앞서 프로그래밍 세계에서 공백조차도 때론 어떤 의미를 갖는다고 언급했었는데 바로 이런 경우의 공백으로서 파이썬 표준문법에 존재하는 의도성을 갖는 공백이다. 함수에서만 쓰이는게 아니라, 이후 다른 여러 코드문치에서 반복해서 사용될 것이니, 처음부터 잘 알아두자. 이 공백의 의미를 간단히 설명하면, 매 라인마다 같은 크기의 공백을 둬으로써 각 라인은 서로 묶여진 한 코드 덩어리임을 나타내기 위한 목적이다.

이를 우리 코드에 빗대어 보면 즉, `when_start`라는 함수에 속한 코드가 어디서부터 어디까지인지를 컴퓨터에게 알려주기 위해 공백을 사용한 것이다. 지금 이 함수는 호출되면 그 안에 딱 1개의 라인만 수행하는 너무 간단한 함수라 이 말이 잘 와닿지 않을 수 있는데, 추후에 더 많은 코드를 수행하는 수행내용이 긴 함수를 보면 더 잘 의미가 와닿을 것이다.

마지막으로 **`Entry.print("Hello World!")`** 을 분석하자. 하나씩 뜯어보면, `Entry`라는 라이브러리 이름을 적었고, 이후에 점(.)을 찍었고, 이후에 `print("Hello World!")` 를 적었다. 앞서 프로그래밍 세계에서 점(.) 하나도 의미가 있다고 했는데, **`Entry.`** 라고 라이브러리 이름에 점을 찍으면, 그 라이브러리 안에 있는 것들 즉 함수(또는 클래스, 변수 등)를 사용하겠다는 의미이고, 그 뒤에 연이어 적인 **`print("Hello World!")`**가 바로 `Entry` 라이브러리 안에 위치한 함수이다. 결국 이 라인에 적힌 코드의 의미는 1번 라인에서 `import`해 가져온 `Entry` 라이브러리 안에 위치한 `print`라는 함수를 호출해 사용하겠다는 의미이다. `print`라고 이름지어진 뜻에서 유추할 수 있듯이, 화면에 어떤 값을 출력하는 용도이고, 출력할 내용은 "(큰따옴표)안에 적음으로써 화면표기될 내용의 시작이 어디고 끝인지를 의미표기해 전달한 것이다.

휴~ 이로써 6라인짜리 아주 간단한 프로그램의 코드분석은 다 끝났다! 속 시원한가? 엔트리의 "시작하기" 버튼을 누르면 저 6라인의 코드가 실행되면서 화면에 우리가 원하는 결과(`Hello World!` 라고 말하기)가 출력되는가? 잘 출력될 것이다. 코드가 다 해석되었고, 실행도 잘된다. 그럼에도 당 신에겐 속 시원하지 않고 뭔가 다 해결되지 않은 의문의 찝찝함이 여전히 남아야 한다. 남았는가? 남지 않았다 하더라도 실망하지 말자.

남아야 하는 의문은 우리의 단지 이 6라인짜리 코드에서 과연 어떻게 원하는대로 동작하는 결과가 나왔는지? 라고 오히려 의아해 해야 하는 것이다. 원하는 결과가 나오기에는 사실상 코드가 완전하지는 않기 때문이다. 분명히 코드가 더 있어야 되는데 말이다. 우리는 `when_start` 함수를 정의만 했지 사용하기 위해 이를 호출한 코드는 어디에도 존재하지 않기 때문이다. 구체적으로 `when_start` 함수를 내 코드 안에 어딘가에서는 호출해야 하고 그래야 그 호출에 의해 함수가 실행되면서 화면출력이 이뤄지는 것인데, 그런데 우리 6라인짜리 프로그램 내 코드 안 어디서든 `when_start` 함수를 직접 호출하는 코드는 없다. 그렇다면 저 함수는 누가 호출해서 어떻게 실행이 된 것이란 말인가??

콜백함수란

답을 말하면, 우리 프로그램 내 코드에서 직접 호출한게 아니라 우리 프로그램 밖의 외부에서 저 함수의 호출이 이뤄진 것이다. 그렇다면 누가?? 그렇지 않다. 호출자는 바로 엔트리 프로그램 그 자신이다. 엔트리가 우리 함수 `when_start`를 자동으로 호출한거다. 여기서 상식으로 알아두면 좋을 용어 하나가 등장하는데 이와같이 함수간 함수인데 내가 직접 호출이 없이 외부에서 어떤 시점에 자동호출되는 함수를 **콜백(callback)**함수라고 부른다. 그렇다, `when_start`는 콜백함수였던 것이다. 이러한 콜백함수는 함수 이름을 우리 맘대로 정할 수 없다. 엔트리 자신이 이미 정해놓은 함수이름을 그대로 사용해야만 하는 것인데 즉, 엔트리 내부적으로 어떤 약속이 되어 있냐면, 사용자가 "시작하기" 버튼을 누르는 즉시, 엔트리는 내부적으로 항상 `when_start` 함수를 호출하기로 코딩되어져 있는 것이다. 혹시 이 말을 믿지 못하겠다면, `when_start` 라는 함수이름을 의도적으로 다른 것으로 바꾼 후, "시작하기" 버튼을 눌러보도록 하자. 기대했던 동작이 이뤄지는가? 정상적인 실행이 되지 않을 것이다. 우리 프로그램 안에는 사전 약속되어진 필수로 존재해야만 하는 바로 그 `when_start` 함수가 존재하지 않기 때문이다.

단지, 6라인짜리 작은 프로그램이었지만, 텍스트 코딩의 많은 핵심을 이해할 수 있는 좋은 예제였음에 틀림이 없다. 다음 단계로 프로그래밍을 구성하는 핵심요소(순차, 반복, 조건 등등)들을 하나 하나씩 격파해 나가도록 하자.

3.2 입/출력 (Input/Output)

프로그램에 입/출력은 무엇인가요? 사용자로부터 필요한 정보를 받아드리거나 반대로 보여주는 방식으로 사용자 인터랙티브(Interactive) 한 의사소통하기 위한 수단일 것이다.

엔트리 블록코딩에서 우리는 이 두 가지를 다음의 두 블록(을 사용해서 입출력을 코딩해 본 적이 있다.



이제 이를 텍스트코딩으로 구현해보자. 입/출력 기능은 모두 우리가 이전 장에서 배운 Entry 라이브러리 안에 있는 두 종류의 함수(print, input)를 호출함으로써 구현할 수 있다. 함수 이름 자체가 무슨 용도인지 알 수 있을만큼 직관적이다. 해당 함수들은 엔트리-파이썬의 경우 실제 파이썬과는 조금 다르게(실제 파이썬에는 라이브러리 호출 없이 곧바로 사용 가능하며, 이러한 함수들을 특별히 언어자체에 기본으로 내장된 함수라 하여 내장 함수라 부름) Entry라는 라이브러리 안에 존재한다. 이 경우 이 함수들을 어떻게 호출해 사용할 수 있는지를 이전 과에서 이미 배웠고, 라이브러리 이름.함수명 이라는 문법형식으로 호출해야 하는 것을 익히 알고 있다. 따라서, **Entry.print()** 와 **Entry.input()** 으로 호출하면 된다. 다만, 함수가 올바르게 동작하기 위해 호출자로부터 넘겨받아야 할 필수값(파라미터)이 무엇인지만 정확히 알면 되겠다.

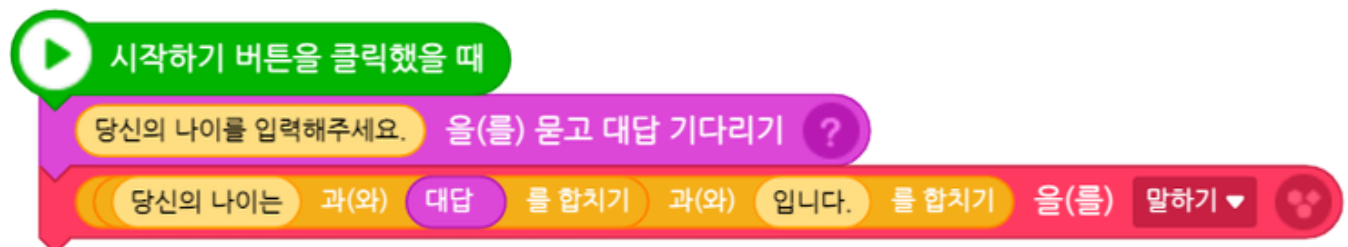
사실 이것도 직관적으로 예상할 수 있듯이 print 함수(출력함수)의 경우, 사용자에게 보여주고자 하는 문구를 넘기고, input(입력함수)의 경우, 사용자로부터 어떤 값을 받아드리고 싶은지 설명하는 문구를 넘겨주면 된다. 이 문구는 사실상 어떠한 문장(텍스트코딩 세계에서는 이러한 데이터의 형식을 스트링(string)이라고 함)이기 때문에 문구를 의미하는 문법표현으로 ""(큰 따옴표) 안에 원하는 내용을 적으면 된다.(작은 따옴표도 되긴 하나 엔트리-파이썬에서는 자동으로 큰 따옴표로 변경된다.) 조금더 재미있는 부분은 출력하고 싶은 문구를 몇개의 문구를 나눈 후 이를 재조립해 출력하는 방법도 가능하다는 것이다. 여러 문구를 조립할 때 문구들 사이에 사용하는 연산자 기호는 + (문구들을 덧붙인다는 직관적 의미를 가짐)이다.

이제 지금까지 이론적으로 배운 부분은 모두 합쳐 사용자의 나이를 입력받고, 출력하는 아주 간단한 프로그램을 텍스트코딩으로 코딩해보자.

실행결과



블록코딩



엔트리-파이썬

```
1 | # Entrybot 오브젝트의 파이썬 코드
2 |
3 | import Entry
4 |
5 | def when_start():
6 |     Entry.input("당신의 나이를 입력해주세요.")
7 |     Entry.print("당신의 나이는" + Entry.answer() + "입니다.")
```

코드를 이해할 수 있겠는가? 조금 낯선 부분이 등장했는데, 바로 `Entry.answer()` 라는 부분이다. 이 함수는 왜 필요할까? 사실 이 함수의 필요는 엔트리-파이썬에만 필요한 함수(실제 파이썬에서는 사용자가 입력한 값을 자신이 직접 정의한 변수에 담아 사용하기 때문에 저런 용도의 함수가 필요하진 않음)로, 함수의 용도는 사용자가 입력한 값을 무엇인지 알아내기 위한 용도로서 `Entry.print` 함수 안에서 출력문장 조립의 한 부분으로서 직접 사용함으로써 사용자의 입력값을 가져다가 출력하는데 활용하였다.

내부적인 동작을 좀더 설명하면 사용자가 어떤 값을 입력하면 그 값을 코드의 다른 부분에서 재활용하기 위해서는 어떻게든 그 값을 어느 공간에 임시로 저장(보통 이를 변수라는 저장공간에 저장하게 됨)을 해두어야 하고, 그 변수를 통해 입력값을 코드의 다른 부분에서 활용하게 된다. 그러나, 우리의 코드는 어디에서 우리가 정의한 변수가 없음에도 원하는 정상동작을 구현했는데, 사실은 엔트리 내부적으로 사용자가 입력한 값을 노출하지 않는 숨겨진 임시변수에 저장해 두고 있고, 우리에게는 단지 그 값을 읽어 올 수 있는 `answer()`라는 함수만을 제공하고 있다.

앞서 언급했듯이 실제 파이썬 코딩에서는 이렇게 활용되지 않는 부분이기도 하나, 엔트리 블록코딩에서 사용자의 편의성을 최대한 끌어올리다보니 불가피하게 발생하는 부분이라고 이해하면 좋겠다. 그리고, 그 블록코딩을 그대로 계승해야 하는 엔트리-파이썬 입장에서조차 불가피한 부분이기도 하다. 엔트리-파이썬은 블록코딩에서 텍스트코딩으로 넘어가기 위한 중간 연결다리로서의 역할이지 파이썬 그 자체를 정확하게 배우기 위한 용도는 아님을 이 예제에서도 알 수 있었다.

3.3 변수 (Variable)

우리는 엔트리 블록코딩에서 변수와 리스트라 불리는 것을 사용해왔다. 둘 다 데이터를 담는 저장공간이었다. 그중에서 먼저 변수는 무엇이며 그 목적은 무엇이었는지를 상기해 보자. 변수는 그 이름 그대로 고정되지 않은 변화하는 값을 임시로 보관해 놓을 공간과 같은 것이다. 우리는 블록코딩에서 어떠한 블록을 사용할 때, 대부분의 블록은 특정한 사전에 정의된 어떤 행동을 유도하기 위해 그에 맞는 고정된 값을 지정해 사용하는게 일반적이다. 하지만, 미리 특정할 수 없는 값들, 예를 들어 사용자로부터 임의로 입력된 값에 기반에 어떤 행동을 하게 만드는 프로그램의 경우도 변수를 사용해야 한다.

방금 언급된 사례인 사용자로부터의 임의의 두 개의 숫자를 받아드려서 합을 구해주는 덧셈만 가능한 아주 간단한 덧셈전용 계산기 프로그램을 변수를 사용해 코딩해보자.

실행결과

당신이 입력한 두 수를 더해 주는 프로그램입니다

first 0
second 0
result 0



대답 0

첫 번째 숫자를 입력하세요.

first 0
second 0
result 0



12

✓

대답 0

두 번째 숫자를 입력하세요.

first 12
second 0
result 0



13

✓

대답 12

입력된 두 수의 합은 25 입니다

first 12
second 13
result 25



대답 13

블록코딩



엔트리-파이썬

```
1 | # 엔트리봇 오브젝트의 파이썬 코드
2 |
3 | import Entry
4 |
5 | first = 0
6 | second = 0
7 | result = 0
8 |
9 | def when_start():
10 |     Entry.print("당신이 입력한 두 수를 더해주는 프로그램 입니다")
11 |     Entry.wait_for_sec(2)
12 |
13 |     Entry.input("첫 번째 숫자를 입력하세요.")
14 |     first = Entry.answer()
15 |     Entry.input("두 번째 숫자를 입력하세요.")
16 |     second = Entry.answer()
17 |
18 |     result = first + second
19 |     Entry.print("입력된 두 수의 합은 " + result + " 입니다")
```

아직 우리가 텍스트코딩에 익숙지 않아서 갑자기 코드량이 많아진 느낌이 들 순 있지만, 사실은 일부러 코딩할 때 공백라인으로 코드 덩어리를 의미 있게 구분해 가독성을 높여놓은 코드라 라인 수가 길어진 느낌은 있지만 차근차근히 해석해보면 복잡한 내용은 거의 없다는 것을 알 수 있다.

그동안 배우지 않았던 첫 등장하는 부분은 5~7라인까지의 3라인에 걸쳐 작성한 코드로, first, second, result라고 이름지은 변수를 3개 선언하고 그 변수값의 초기값(맨 처음 생성하면서 부여한 값)으로 각각에 0을 넣어놓은 것이다. 변수를 만드는 파이썬 문법은 다음과 같다.

변수명 = 초기값

여기서 변수명은 내가 필요한 임의의 이름을 부여하되 그러나 가능하면 의미있게 부여해서 코드를 읽었을 때 쉽게 사용 목적을 파악하도록 하는게 잘 이름짓는 방법이다. 위의 예제에서는 사용자의 첫 입력값(first), 두번째 입력값(second), 그리고 두 값을 더한 결과값(result)이란 의미를 부여 하기 위해 저러한 변수명을 의도적으로 사용했고, 변수값에 처음 생성하면서 동시에 어떤 임의의 값(이를 초기값이라 부름)을 할당해 값을 가진상태 에서 출발되도록 한다. 보통 일반적으로 0 을 초기값으로 사용한다. 해당 문법에서 주의해야 할 것이 하나 있는데 '=' 이라는 기호의 의미는 우리가 수학에서 의미했는 '값이 같다' 라는 의미가 아니라, '=' 를 중심으로 오른쪽에 있는 값을 왼쪽에 할당한다는 의미라는 것을 반드시 기억해야 한다.

¹²₃₄ 따라서, 5번라인의 `first = 0` 이라는 코드는 '='의 왼쪽에 위치한 `first`라는 변수에 '='의 오른쪽에 위치한 `0` 값을 저장한다 라는 의미가 되는 것이다.

[!NOTE]

변수명은 임의로 사용할 수 있다고는 했으나, 사실은 파이썬에서 변수이름을 지을 때, 크게 지켜야 할 2가지 규칙은 있다.

- 변수의 이름은 숫자로 시작할 수 없다.
- 변수의 이름 사이에 공백이 있을 수 없다.

여기서, 참고로 좌우의 값이 같냐라는 수학적 의미를 코딩으로 표현하려면 어떤 기호를 써야하는지 궁금할 수 있는데 이 때는 '==' 기호로 표현하는데 이렇게 '='기호를 한번이 아닌 연달아 두번을 사용해야만 좌우의 값이 서로 같은지 여부의 비교의 의미가 된다.

¹²₃₄ 10~11번 라인 코드덩어리는 프로그램에서 사용자에게 우리 프로그램은 어떤 프로그램임을 알리기 위한 설명을 전달하는 목적으로 10번 라인의 (화면)출력함수는 이미 사용해 보았고, 11번 라인에 `Entry.wait_for_sec(2)` 코드는 처음 등장했는데, 의미해석은 어렵지 않은데 `Entry`라이브러리 안에 있는 `wait_for_sec`함수를 호출해 사용한 건데, 용도는 블록코딩에서의 "~초 기다리기" 블록과 동일한 것으로, 몇 초 기다릴지의 값을 함수로 넘겨주어야 하고, 여기서는 이전 10번 라인에서 출력했던 말을 2초간 화면에 유지하며 기다리게 하기 위해 숫자 2 값을 함수로 넘겨주었다.

¹²₃₄ 13~16번 라인의 코드덩어리는 사용자로부터의 임의의 두 수를 차례차례 입력받아 각각의 두 변수(`first`, `second`)에 저장해 두는 것이고, 이 때 왜 사용자 입력값을 `Entry.answer()`를 통해 값을 읽어 저장해야 하는지는 이전 장 [입출력 함수](#) 부분에서 설명하였다.

¹²₃₄ 18번 라인에서는 두 변수(`first`, `second`)에 저장된 값을 읽어 서로 더한 결과를 세 번째 변수인 `result`에 저장해 두는 것이다.

¹²₃₄ 19번 라인에서는 사용자가 입력한 두 수의 합계가 저장되어 있는 변수 안에 값을 사용자에게 보여줌으로써 프로그램의 목적을 달성하며 끝난다.

우리가 블록코딩에서 이미 변수를 사용해 봤다면, 그 의미와 용도를 잘 알고 있을 것이고 텍스트코딩으로 넘어오더라도 그 의미와 목적이 달라지는 건 전혀없고, 다만 텍스트코딩에서 사용할 때 사용하는 텍스트코딩용 언어(여기서는 파이썬)에 맞는 변수사용 문법을 따라 사용하면 되는 것이다.

3.4 조건 (Condition)

블록코딩에서 사용빈도 매우 높은 블록으로 "만일 ~이라면" 과 "만일 ~이라면, 아니면~" 의 두 가지 블록이 있다. 용도는 우리의 코드가 순차적으로 실행해 나갈 때, 어떤 조건에 따라 실행을 할 수도 있고, 하지 않을 수도 있는 경우에 따른 동작을 만들기 위한 용도라 할 수 있다.

이전 장에서 만들어 오고 있는 계산기 프로그램에서 사용자로부터 입력된 임의의 두 값만을 더해주는 것 외에 사용자에게 더 나은 사용성을 주기 위해 덧셈 뿐 아닌 뺄셈도 가능하고, 둘 중 어떤 계산을 선택할지를 사용자의 선택권에 맡기는 더 나은 프로그램으로 발전시켜보자. 이처럼 소프트웨어를 만드는 일을 한번의 단회성의 제작으로 끝나는게 아니라, 계속 진화해나가면서 더 나은 프로그램으로 버전업이 되어 가는 것이 일반적이며 이를 소프트웨어의 생명주기라고 할 수 있다.

실행결과

당신이 입력한 두 수의 덧셈
또는 뺄셈을 할 수 있는
계산기입니다

first 0

second 0

답 0



첫 번째 숫자를 입력하세요.

first 0

second 0

답 0

13

✓



두 번째 숫자를 입력하세요.

first 13

second 0

답 13

12

✓



덧셈계산을 원하시면 '+'를
입력하고, 뺄셈계산을 원하
시면 '-'를 입력하세요.

first 13

second 12

답 12

-

✓



입력된 두 수의 차는 1 입
니다

first 13

second 12

답 1



블록코딩(1)

시작하기 버튼을 클릭했을 때

당신이 입력한 두 수의 덧셈 또는 뺄셈을 할 수 있는 계산기 입니다 을(를) 말하기 ▼

2 초 기다리기

첫 번째 숫자를 입력하세요. 을(를) 묻고 대답 기다리기 ?

first ▼ 를 대답 (으)로 정하기 ?

두 번째 숫자를 입력하세요. 을(를) 묻고 대답 기다리기 ?

second ▼ 를 대답 (으)로 정하기 ?

덧셈계산을 원하시면 '+'를 입력하고, 뺄셈계산을 원하시면 '-'를 입력하세요. 을(를) 묻고 대답 기다리기 ?

만일 대답 = + (이)라면

입력된 두 수의 합은 과(와) first ▼ 값 + second ▼ 값 를 합치기 과(와) 입니다 를 합치기 을(를) 말하기 ▼

만일 대답 = - (이)라면

입력된 두 수의 차는 과(와) first ▼ 값 - second ▼ 값 를 합치기 과(와) 입니다 를 합치기 을(를) 말하기 ▼

엔트리-파이썬(1)

```
1 | # 엔트리봇 오브젝트의 파이썬 코드
2 |
3 | import Entry
4 |
5 | first = 0
6 | second = 0
7 |
8 | def when_start():
9 |     Entry.print("당신이 입력한 두 수의 덧셈 또는 뺄셈을 할 수 있는 계산기 입니다")
10 |    Entry.wait_for_sec(2)
11 |
12 |    Entry.input("첫 번째 숫자를 입력하세요.")
13 |    first = Entry.answer()
14 |    Entry.input("두 번째 숫자를 입력하세요.")
15 |    second = Entry.answer()
16 |
17 |    Entry.input("덧셈계산을 원하시면 '+'를 입력하고, 뺄셈계산을 원하시면 '-'를 입력하세요.")
18 |    if Entry.answer() == "+":
19 |        Entry.print("입력된 두 수의 합은 " + (first + second) + " 입니다")
20 |    if Entry.answer() == "-":
21 |        Entry.print("입력된 두 수의 차는 " + (first - second) + " 입니다")
```

블록코딩(2)



엔트리-파이썬(2)

```
1 | # 엔트리봇 오브젝트의 파이썬 코드
2 |
3 | import Entry
4 |
5 | first = 0
6 | second = 0
7 |
8 | def when_start():
9 |     Entry.print("당신이 입력한 두 수의 덧셈 또는 뺄셈을 할 수 있는 계산기 입니다")
10 |    Entry.wait_for_sec(2)
11 |
12 |    Entry.input("첫 번째 숫자를 입력하세요.")
13 |    first = Entry.answer()
14 |    Entry.input("두 번째 숫자를 입력하세요.")
15 |    second = Entry.answer()
16 |
17 |    Entry.input("덧셈계산을 원하시면 '+'를 입력하고, 뺄셈계산을 원하시면 '-'를 입력하세요.")
18 |    if Entry.answer() == "+":
19 |        Entry.print("입력된 두 수의 합은 " + (first + second) + " 입니다")
20 |    else:
21 |        if Entry.answer() == "-":
22 |            Entry.print("입력된 두 수의 차는 " + (first - second) + " 입니다")
```

기존 프로그램에서는 변수 3개(first, second, result)를 사용했다면, 이번 프로그램에서는 변수 2개(first, second)만 사용하도록 개수를 줄였는데, 줄이는 방법은 기존에는 두 값의 계산결과를 다시 result라는 변수에 담아서 그 변수에 담긴 값을 출력하는 식으로 했다면, 이번에는 해당 변수 없이 (first + second), (first - second)라고 계산식을 괄호와 함께 사용함으로써 출력함수(print함수) 안에서 직접계산하여 결과 출력하도록 하였다.

34 17번 라인을 살펴보자. 우리는 사용자에게 '+' 또는 '-' 둘 중에 하나를 선택해 입력는 선택권을 주도록 안내했기 때문에 사용자가 둘 중에 하나의 원하는 연산종류를 정상적으로 입력한다고 가정하고, 사용자가 선택한 연산종류에 따라 코드실행을 달리해야하기 때문에 조건문을 사용했는데 파이썬 언어에서 조건문을 사용하는 문법은 다음과 같다.

if 조건:

조건이 만족(또는 참)일 때 실행할 내용들
공백 4칸 또는 탭 1칸

34 위에 문법에 빗대어 18-19번 라인의 다음의 코드를 이해할 수 있는지 확인해 보자.

```
18 | if Entry.answer() == "+":  
19 |     Entry.print("입력된 두 수의 합은 " + (first + second) + " 입니다")
```

이전 장에서 '==' 기호(연산자)의 의미가 무엇인지 설명했다. 좌우의 두 값이 같은지를 확인하기 위한 연산자(operator)로서 사용자가 입력한 값이 '+' 였는지를 확인하고 있다. 만약 사용자의 입력값이 '+' 라면 그렇다면, if 로 시작한 문장의 다음에 위치한 코드들이 실행될 것이다. 여기서 코드작성시 주의해야 할 것은 우리가 [Hello World 예제코드 분석](#)에서 알아본 바와 같이 if 문 다음에 의도적인 공백(일반적으로 키보드의 스페이스바 로 4 칸 또는 탭으로 1칸을 누름)을 만들어 매 라인마다 같은 크기만큼 공백을 둬으로써 if 조건이 만족에 경우의 수행할 코드덩어리의 시작과 끝이 어디 인지를 분간할 수 있게 해주어야 한다는 것이다. 따라서, 공백을 정확하게 입력해야 한다는 것을 잘 숙지해야 하겠다.

34 위에 조건문의 사용이 정확하게 이해되었다면, 그 다음 20-21번 라인의 코드는 당연히 쉽게 이해될 것이다.

마지막으로, 위에 예제코드를 블록코딩(1과 2), 텍스트코딩(1과 2)로 구분해 놓은 이유를 설명해야 할 것인데, 1과 2의 두 종류의 코드는 모두 우리가 목표한 동일한 기능을 하되, 다만, 구현방법에 있어서 약간 다른 방식으로 구현한 코드로서 지금 배운 조건문 외 한 가지 더 추가적인 조건문 사용문법을 설명하기 위한 의도가 있었다.

우리가 블록코딩에서 "만일 ~이라면" 이라는 블록의 텍스트코딩에서는 어떻게 표현하는지를 지금까지 배웠다면, "만일 ~이라면, 아니면~" 이란 블록의 사용법도 텍스트코딩에도 존재함을 미루어 짐작할 수 있는데, 이를 문법사용법은 다음과 같다.

if 조건:

조건이 만족(또는 참)일 때 실행할 내용들
공백 4칸 또는 탭 1칸

else :

조건이 만족하지 않을(또는 거짓) 때 실행할 내용들
공백 4칸 또는 탭 1칸

```
18 | if Entry.answer() == "+":  
19 |     Entry.print("입력된 두 수의 합은 " + (first + second) + " 입니다")  
20 | else:
```

```

21 |     if Entry.answer() == "-":
22 |         Entry.print("입력된 두 수의 차는 " + (first - second) + " 입니다")

```

위에 코드는 아래와 같이 블록코딩과 비교해보면 더 쉽게 이해할 수 있다.

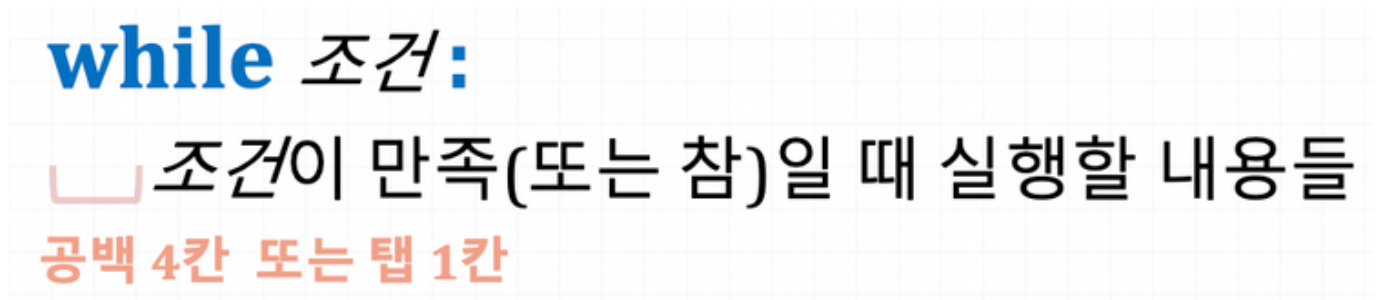


다만, if~else~ 형식의 텍스트코딩의 조건문의 사용상에 있어서도 주의해야할 부분은 이전과 마찬가지로 각 코드의 라인마다 동일한 공백을 길이를 얼마만큼 유지해야만 하는지에 주의를 기울여 코딩해야 한다는 것이다. 블록코딩에서는 비주얼적으로 조건문에 시작과 끝이 명확하게 감싸져있는 방식으로 표현해서 우리가 직관적으로 이해가 가능했지만, 텍스트코딩에서는 텍스트로만 이를 표현해야하는 한계성 안에서 블록코딩 표현상 감싸져 있는 코드덩어리는 텍스트코딩에서는 매 라인마다 동일 공백량의 길이조절을 통해(들여쓰기를 통해) 이를 표현하고 있음을 이해할 수 있다.

3.5 반복 (Loop)

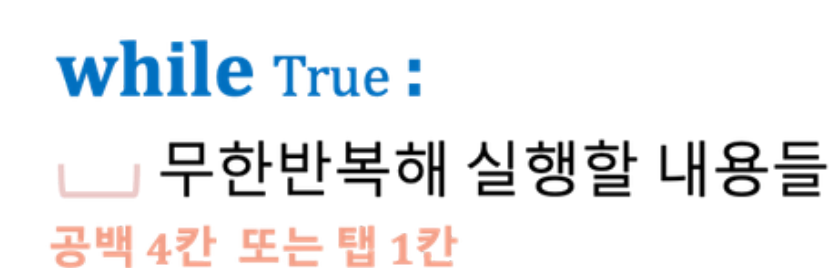
우리는 블록코딩에서 코드의 흐름(flow)을 제어하는 목적(따라서, 엔트리의 블록 분류 상에서 '흐름' 카테고리 안에 존재)의 블록 중 가장 기본이 되는 블록 중에 하나로 "반복"이라는 것을 무수히 사용해 본 경험이 있다. 반복수행해야 하는 코드덩어리를 어떻게 얼마만큼 반복해야 하는지에 따라 엔트리 '흐름' 카테고리 안에 반복과 관련된 블록을 보면 크게 세 가지 종류인데, "무한반복 / 지정한 횟수만큼 반복 / 어떤 조건을 만족하는 동안의 반복" 이다.

먼저, "어떤 조건을 만족하는 동안의 반복" 파이썬 문법으로 표현하면 다음과 같다.



while이라는 영어접속사의 의미 자체도 "~하는 동안" 뜻이고, 결국 while 문 안에 적힌 '조건'이 만족되는 동안하는 그 while 문 아래 기술된 내용을 계속 반복수행한다는 의미이다. 그 말을 이렇게 가정할 수 있는데 반복을 수행하는 동안 '조건'이 만족하는지 안하는지를 매번 확인한다는 것이고, 그 사이 조건이 변화해서 '조건'을 만족하지 않는 어떤 시점이 올 수 있고, 그 시점에 도달 했을 때는 반복을 그만두고, while문을 빠져나와 while 문 다음으로 이어진 문장의 코드가 순차적으로 수행되게 될 것이라는 의미이다.

우리는 일반적으로 반복을 수행하는 동안 '조건' 이 변화해 어느 시점에는 '조건'이 만족하지 않아 반복을 빠져나와야 할 것으로 기대하는데, 만약 반복을 의도적으로 무한히 하고 싶다면, 어떻게 해야할까? 그것은 '조건' 그 자체를 결코 빠져나올 수 없는 조건으로 의도적으로 생성해 주면 되는 데 그렇다 '조건'을 강제로 항상 참(True)이 되도록 강제할당하면 될 것이다. 이를 파이썬 텍스트코딩 문법으로 표현하면 이렇게 된다.



방금 배운 테크닉을 이용해 이전 장에서 만들어 오고 있는 계산기 프로그램을 더욱더 발전시켜 사용자의 사용성을 개선시켜 나가보자. 사용자 입장에서는 계산기의 계산을 딱 한번의 단회성으로 끝내지 않고, 연이어 계속하려면 현재로서는 프로그램(또는 앱)을 재실행시키는 것 외에는 방법이 없다. 이는 사용성에 제약이 큰 것으로 사용자 본인이 원하는 만큼 계산을 계속하고, 더 이상 필요가 없을 때 프로그램을 종료할 수 있게 해주는 것이 필요하다. 이러한 사용자 요구사항에 맞춰 프로그램을 개선하기 위해 우리는 우리의 코드에 반복문을 추가할 필요가 있다.

실행결과

덧셈계산을 원하시면 '+'를
입력하고, 뺄셈계산을 원하
시면 '-'를 입력하세요.
앱을 종료하고 싶으시면 'x'
'를 입력하세요.



대답 5

first 3

second 5

블록코딩

```

시작하기 버튼을 클릭했을 때
  당신이 입력한 두 수의 덧셈 또는 뺄셈을 할 수 있는 계산기입니다. 을(를) 말하기
  2 초 기다리기
  계속 반복하기
    첫 번째 숫자를 입력하세요. 을(를) 묻고 대답 기다리기
    first 를 대답 (으)로 정하기
    두 번째 숫자를 입력하세요. 을(를) 묻고 대답 기다리기
    second 를 대답 (으)로 정하기
    덧셈계산을 원하시면 '+'를 입력하고, 뺄셈계산을 원하시면 '-'를 입력하세요. 앱을 종료하고 싶으시면 'x'를 입력하세요. 을(를) 묻고 대답 기다리기
    만일 대답 = x (이)라면
      모든 코드 멈추기
    만일 대답 = + (이)라면
      입력된 두 수의 합은 과(와) first 값 + second 값을 합치기 과(와) 입니다 를 합치기 을(를) 말하기
    만일 대답 = - (이)라면
      입력된 두 수의 차는 과(와) first 값 - second 값을 합치기 과(와) 입니다 를 합치기 을(를) 말하기
  2 초 기다리기
  
```

엔트리-파이썬

```
1 | # 엔트리봇 오브젝트의 파이썬 코드
2 |
3 | import Entry
4 |
5 | first = 0
6 | second = 0
7 |
8 | def when_start():
9 |     Entry.print("당신이 입력한 두 수의 덧셈 또는 뺄셈을 할 수 있는 계산기 입니다")
10 |    Entry.wait_for_sec(2)
11 |
12 |    while True:
13 |        Entry.input("첫 번째 숫자를 입력하세요.")
14 |        first = Entry.answer()
15 |        Entry.input("두 번째 숫자를 입력하세요.")
16 |        second = Entry.answer()
17 |
18 |        Entry.input("덧셈계산을 원하시면 '+'를 입력하고, 뺄셈계산을 원하시면 '-'를 입력하세요. 앱을 종료하고 싶으시면 'x'를 입력")
19 |        if Entry.answer() == "x":
20 |            Entry.stop_code("all")
21 |        if Entry.answer() == "+":
22 |            Entry.print("입력된 두 수의 합은 " + (first + second) + " 입니다")
23 |        if Entry.answer() == "-":
24 |            Entry.print("입력된 두 수의 차는 " + (first - second) + " 입니다")
25 |        Entry.wait_for_sec(2)
```

이제 작성한 코드를 이해해보자. 개선된 계산기 프로그램(또는 앱)에서 무한반복해야 할 코드들은 while True: 다음 라인부터에 위치하여 전체적으로 안쪽으로 들여쓰기해 적은 것을 확인할 수 있다. 사용자가 우리의 계산기앱을 종료하고 싶을 때는 사용자 안내문구에서 요구된 것처럼 'x'라는 값을 입력했을 때이고, 이 부분의 처리는 코드 19-20라인에서 만약 사용자로부터 입력된 값이 'x'일 경우에 Entry 라이브러리 안에 있는 앱 종료함수인 stop_code함수를 호출하되 앱상의 모든 코드를 종료하기 위해 "all"이라는 아규먼트(인자)값을 넘겨서 호출했음을 알 수 있다.

3.6 리스트 (List)

블록코딩에서 리스트라고 불렀던 저장공간의 목적은 무엇이었는가? 앞에서 변수가 단 하나의 변화하는 값(또는 사전에 값을 특정할 수 없는 값)을 저장했다면, 리스트라는 공간은 변화하는 단 하나의 값이 아니라, 여러 개의 복수의 값을 저장할 수 있는 있는 공간인 것이다.

이러한 공간을 파이썬에서는 리스트(list)라고 부르고, 다른 텍스트코딩 언어에서는 일반적으로 배열(array)라고 부르는데 엔트리에서 이러한 복수 개의 값을 저장하는 공간을 이름지으면서 어떤 다른 단어로 예를들어 '목록' 등의 번역된 한글이름으로 표현할 수 있었음에도 의도적으로 리스트(list)라고 파이썬 상의 용어표현 그대로 도입한 것부터 사실상 이 개념을 이해시키기 위한 잘짜여진 각본이 있었던 생각이 든다.

블록코딩에서 리스트가 엔트리-파이썬 텍스트코딩에서는 어떻게 사용할 수 있을까? 그럼, 예제를 통해 엔트리-파이썬에서 어떻게 코딩할 수 있는지 상세히 알아보도록 하자. 3.3장에서 만들었던 덧셈전용 계산기 프로그램에서 이 리스트를 활용하여 사용성을 더 개선시킬 수 있다. 3.3장의 덧셈전용 계산기는 사용자로부터 입력된 임의의 두 값만을 받아 합계를 구해주고 있다. 이제는 사용자가 입력할 수 있는게 단지 두개의 값만이 아닌 제한 없이 더 많은 수를 연속적으로 입력받아 입력한 모든 수를 더한 합계를 구해줄 수 있다면 사용자 입장에서는 더 나은 사용성을 제공하는 프로그램이 될 것이다.

먼저, 리스트를 어떻게 사용할 수 있는 문법을 확인해보자.

리스트명 = [문자, 숫자 등의 여러 값. 단, 값들은 콤마(,)로 구분]

리스트의 이름은 변수의 이름을 지을 때와 동일하게 임의로 지을 수 있는데, 나를 포함해 누구라도 코드의 의미를 빠르고 효과적으로 이해할 수 있게 하기 위해 항상 의미있게 이름 짓는 습관을 드리는게 필요하다. 이전에도 언급했지만, 이는 프로그래밍 세계에서의 주요한 기본 덕목 중에 하나라고 할 수 있다.

[!NOTE]

리스트명은 임의로 사용할 수 있다고는 했으나, 사실은 파이썬에서 리스트 이름을 지을 때, 변수이름 지을 때와 마찬가지로 크게 지켜야 할 2가지 규칙은 있다.

- 리스트의 이름은 숫자로 시작할 수 없다.
- 리스트의 이름 사이에 공백이 있을 수 없다.

다음은 문법에 맞게 작성한 리스트의 예제들이다.

```
words = ["apple", "banana", "grape"]
```

```
numbers = [1, 3, 10, 25, 2]
```

```
complex = ["apple", 2, 32, "berry", 56, 5]
```

words라는 이름에 리스트는 문자형(string) 데이터 유형(type)의 값(또는 아이템이라 호칭) 3개를 갖고 있다.

numbers라는 이름에 리스트는 숫자형 데이터 유형의 아이템 5개를 갖고 있다.

complex라는 이름에 리스트는 문자형, 숫자형의 데이터 유형의 값을 섞어서 총 6개의 아이템을 갖고 있다.

지금까지 우리는 리스트를 선언하면서 동시에 초기값(선언하면서 동시에 맨 처음 할당하는 값)을 설정하는 법을 배웠다. 이제 더 배워야 할 지식은 리스트 안에 저장된 값들(아이템들)을 접근해 읽어오는 법, 기존 리스트에 새로운 신규 값(아이템)을 추가하거나, 삭제하는 방법에 대해 알 필요가 있다. 방금 언급된 리스트를 다루는 방법들은 이번 장의 서두에서 언급된 우리가 목표하는 앱인 발전된 덧셈계산기 앱의 코드들을 하나씩 분석해 보면서 배워나가기로 하자.

실행결과

더할 숫자를 입력하세요. 현재까지 입력한 값의 합을 알고 싶으면 '='를 입력하세요. 앱을 종료하고 싶으시면 'x'를 입력하세요.



numbers	
0	2
1	3

result 0

=

✓

5



numbers	
---------	--

result 5

블록코딩



엔트리-파이썬

```
1 | # 엔트리봇 오브젝트의 파이썬 코드
2 | import Entry
3 |
4 | result = 0
5 | numbers = []
6 |
7 | def when_start():
8 |     Entry.print("당신이 입력한 모든 수를 더해주는 계산기 입니다")
9 |     Entry.wait_for_sec(2)
10 |
11 |     while True:
12 |         Entry.input("더할 숫자를 입력하세요. 현재까지 입력한 값의 합을 알고 싶으면 '='를 입력하세요. 앱을 종료하고 싶으시면 'x'")
13 |
14 |         if Entry.answer() == "x":
15 |             Entry.stop_code("all")
16 |
17 |         if Entry.answer() == "=":
18 |             while len(numbers) != 0:
19 |                 result = result + numbers[0]
20 |                 numbers.pop(0)
21 |
22 |             Entry.print(result)
23 |             Entry.wait_for_sec(2)
24 |             result = 0
25 |         else:
```

34 먼저 5번라인에서 numbers라는 이름에 리스트 한개를 선언한 내용이 좀 특이한데, 초기값이 전혀 없는 상태로 리스트를 생성했다는 것을 알 수 있는데, 왜냐하면 대괄호 안에 안에 아무값도 없는 상태로(그냥 대괄호만 열고 곧바로 닫는 형태로) 선언했기 때문이다. 이렇게 선언한 이유는 남득이 가능한데, 우리 앱은 사용자가 임의로 제한없이 입력하는 숫자값들을 담아놓을 임시공간이 필요하고, 그러한 목적에 부합하는 공간인 리스트를 활용하되 처음에는 사용자가 아무것도 입력하지 않은 상태이고, 이 상태에서도 리스트의 존재는 필요함으로 이런식으로 초기화 했다고 예상할 수 있다.

34 다음으로 26번 라인을 먼저 살펴보자. 26번 라인은 리스트에 사용자가 입력한 숫자값을 신규 아이템으로 추가하는 코드이다. `append`라는 함수를 통해 사용자 입력값을 `Entry.answer()` 통해 읽어 전달하고 있는데 그 `append` 함수의 사용을 위해 **number.**으로 즉, 리스트의 이름 안에 위치한다는 의미로 함수를 호출하고 있는 점이 특이할 것이다. 어떻게 이런식의 함수호출이 가능할까? 이유는 리스트 그 자체가 객체(Object)이기 때문에 그런 것인데, 이 부분을 자세히 설명하기에는 이 책의 범위를 초과하는 것이라 따로 관심있는 사람은 따로 파이썬에서의 객체 다루기에 대해 공부하길 바란다. 현재로서는 리스트에 신규 아이템 추가 방법은 리스트이름.**append**(추가하려는 값) 문법을 통해 사용한다고 알아두면 되겠다.

34 이제 18~20번 라인을 살펴보자. `while`을 사용한 반복문이다. 그런데 언제까지 반복할지에 대한 조건이 있는데, `len(numbers) != 0` 라는 조건이고, 여기서 우리는 기존에 "`==`" 기호(연산자)가 의미하는 것이 무엇인지는 알고있다. 하지만, "`!=`"의 의미는 무엇일까? 바로 "`==`"의 반대 의미로, 좌우의 두 값이 서로 같지 않을 때를 나타내는 의미가 된다. 그렇다면 이 코드의 의미를 해석하면 `len(numbers)` 함수의 리턴값이 0 과 같지 않을 때까지 반복하는 것이다.

이제 `len` 함수가 무엇인지 알아야겠다. 사용법을 보면 특이한데 함수명 앞에 그 함수가 위치한 어떤 위치정보 예를들어 어떤 라이브러리의 이름이 나, 리스트명나 변수명으로 참조하여 함수를 부른 것이 전혀없다. 이 말인 즉은 이 함수는 어디에 속한게 아닌 파이썬 언어 내부에 디폴트로 내장되어 있는 함수라는 것이다. 그래서, 이름만으로 맘대로 불러서 사용할 수 있는 특별함수이고 우리는 이를 특별히 내장함수라고 호칭한다. 그렇다 `len` 함수는 내장함수이고, 용도는 그 이름에서 유추할 수 있는데, `length`(길이)의 약자로 만든 함수 이름으로 결국 특정 데이터가 보유하고 있는 데이터의 양(데이터의 길이)을 알려주는 함수이다. 따라서, 여기서 `len` 함수로 넘겨진 데이터값은 `numbers`라는 리스트로 결국, 해당 리스트가 갖고 있는 아이템의 개수를 알려주게 된다. 26번 라인의 코드에 의해서 사용자가 임의의 숫자 2개를 입력했으면, `len` 함수로 확인해 보면 현재 아이템개수는 2라는 것을, 사용자가 임의의 숫자 3개를 입력했으면 현재 아이템개수는 3이라는 것을 알 수 있게 된다.

위에서 각각 개별적으로 파악된 의미들을 종합해 해당 코드를 정확하게 다시 해석하면 ***len(numbers) != 0*** 이란 조건을 가진 ***while*** 문은 결국, ***numbers*** 라는 리스트에 ***아이템이 하나도 없을 때까지 반복***하라는 것이다. 사용자가 임의로 입력하는데로 numbers에는 계속 아이템들이 채워질 텐데 어쩌서 하나도 없는 상황이 가능한 것일까 라는 의문이 들 수 있다. 그것은 19~20번 라인의 코드를 해석하면 이유를 찾을 수 있다.

34 19~20번 라인의 코드는 사용자에게 의해 입력된 numbrers 리스트 안에 숫자값들을 하나하나씩 꺼내와 합계를 구하는 코드이다. 위에서 우리는 리스트를 다루는 방법 3가지(신규 값을 넣기, 기존 값을 읽기, 기존 값을 제거하기) 중에 1가지(신규 값을 넣기)를 배웠고, 이번엔 기존에 저장되어 있는 읽어들 수는 방법을 배울 차례이다. 이를 정확히 이해하려면 먼저, 문법과 더불어 리스트의 참조 인덱스(reference index)라는 개념에 대해 배워야 한다. 리스트의 특정 아이템을 읽는 문법은 다음과 같다.

리스트이름[참조 인덱스]

다음 예에서 볼 수 있듯이 리스트 안에 아이템이 존재하게 되는 그 순간(리스트 정의와 동시에 초기값 설정이든, 추후 신규 아이템이 추가되든), 곧바로, 다음에 그림과 같이 파이썬 언어 내부적으로 각 아이템이 리스트 내에 위치한 순서에 따라 맨 앞에서부터 0부터 1씩 증가하며 자동으로 매겨지는 우리 눈에는 보이지 않는 인덱스값이 내부적으로 존재하게 된다. 우리는 그 인덱스값을 이용해서 리스트 안의 각 아이템의 값들을 읽을 수 있게 된다.

```
words = ["apple", "banana", "grape"]
참조 인덱스:      0           1           2
```

우리가 만약 저 words라는 리스트 안에 두번째 위치한 "banana"라는 아이템(값)을 읽어오고 싶다면, 이제 리스트의 특정 위치의 아이템을 읽는 문법에 따라 words[1]이라고 표현해 읽을 수 있는 것이다. 아래 그림처럼 엔트리는 이러한 리스트 내의 값들과 참조 인덱스값들을 실시간으로 시각

화하여 보여주고 있는데, 우리의 예제에서 실행결과화면을 보면 좌측상단에 numbers 리스트 안에 2와 3이 순서대로 들어있는 것을 확인할 수 있다. 즉 사용자는 처음에 2라는 숫자값을 입력하고, 이어서 3이라는 숫자값을 입력한 상황인 것이다.



12 34 19라인의 코드는 `numbers[0]`이라는 문법표현으로 `numbers` 리스트 안에 맨 첫번째 아이템을 읽어서 그 값을 `result`라는 변수에 저장해 놓는 것이다. 바로 다음의 20번 라인 코드는 무엇인가? `numbers` 리스트 안에 위치한 `numbers.pop()`의 형식으로 `pop` 함수를 부르고 있는데, `pop`이라는 그 이름에서 알 수 있듯이 리스트 안에 특정위치의 데이터를 꺼내서 삭제하는 용도의 함수이다. 그래서, 이 함수의 사용은 **리스트이름.pop(삭제하려는 아이템의 참조 인덱스)** 형태로 호출해 사용할 수 있다. 따라서, 20번 라인의 `numbers.pop(0)`의 의미는 리스트 안에 맨 첫번째(인덱스 0에 의해) 아이템을 리스트로부터 삭제(제거)하라는 의미이다.

우리 예제의 위의 실행 결과화면에서(2, 3값이 연달아 입력되어진 상태에서) 연이어 살펴보면, 이 코드가 실행되면 리스트 안에 맨 처음값 숫자 2는 리스트에서 제거됨과 동시에 그 뒤에 위치해 있던 숫자 3이 방금 제거되어진 2가 있던 위치로 자동으로 끌어당겨져와 이제 맨앞에 위치하게 된다. `pop(0)`라는 의미는 결국 현 시점에서 항상 리스트 상에 맨 앞에 위치한 값을 가르키게 되는 것이다. 위에 코드를 엔트리에서 직접 실행시켜보자. 글로 읽은 것보다 한번 보는게 더 나운데 엔트리가 이러한 단계적 동작을 실시간으로 시각화해 비주얼적으로 보여주므로 직관적으로 이해할 수 있게 되기 때문이다.

12 34 예제 18-20번 라인의 코드를 통합적으로 다시 재해석하면, 현재 `numbers` 리스트에 저장되어 있는 아이템값을 하나씩 꺼내서 읽어 기존 합계(`result`변수값)에 더한 후에 그 읽은 값은 이제 리스트에서 제거해 버리는 일을 하되, 이런식으로 읽고 더하고 제거하는 일을 결국 리스트에 아무 아이템도 남지 않을 때까지 반복해서 수행하라는 의미이다. 즉, 사용자가 입력한 모든 값의 최종 합계를 구하고자하는 목적이며, 합계를 구하면서 동시에 리스트 안에 내용을 비워 깨끗하게 만드는 일을 함께 수행하고 있다.

다시 처음으로 돌아가 우리가 목표한 사용자로부터 제안없는 수를 연속적으로 입력받아 입력한 모든 수를 더한 합계를 구해주는 기능을 갖는 앱을 최종구현 완료하였다. 해당 기능은 반드시 위에서 언급된 식으로만 코딩해야 하는 것이 당연히 아니다. 동일 기능을 여러 가지 방법으로 구현할 수 있는데, 지금의 코드는 그 중에 한 방법일 뿐이다. 사실은 이게 엔트리-파이썬이 아니고, 오리지널 파이썬였다면 훨씬 더 간단하고 효과적으로 구현할 수 있는 코드가 존재하나, 우리는 엔트리-파이썬이 허용하는 문법표현 한계 안에서 코딩할 수 밖에 없고 또 여러분에게 리스트를 조작하는 다양한 표현법을 보여주려고 일부러 저렇게 코딩한 이유도 있다는 것을 참고하길 바란다.

3.7 무작위 수 (Random)

무작위 수(또는 램덤)를 활용하는 것은 단순히 한번의 함수호출이라서 지금까지 내용을 잘 숙지해 따라온 사람에게는 하나도 어려울 것이 없다. 그래서, 이번 장에서는 단순하면서 재미있는 게임하나를 무작위 수 기능을 활용해 만들어 보자.

게임제목은 "내 마음 속의 숫자를 맞춰봐!" 이고, 컴퓨터가 1~50까지의 무작위 수 하나를 생각할 것이고, 우리는 그 수가 무엇인지 최대한 적은 회수의 시도 안에 맞추면 되는 것이다. 게임의 흥미를 높이기 위해 우리가 정답을 맞추기 위해 시도한 답변숫자가 정답에 얼마나 가까운지에 대한 힌트를 지속적으로 주도록 만들었다.

실행결과

kom_num29

my_num0

try_total0

대답0

마음 속으로 1부터 50까지의 수를 생각했으니, 한번 맞춰봐! 몇 번만에 맞출 수 있을까?



31

✓

kom_num29

my_num31

try_total1

대답31

31 보단 더 낮은 수야



kom_num29

my_num29

try_total3

대답29

Bingo! 3 번 만에 찾아냈군!



블록코딩



엔트리-파이썬

```
1 | # 엔트리봇 오브젝트의 파이썬 코드
2 | import Entry
3 |
4 | com_num = 0
5 | my_num = 0
6 | try_total = 0
7 |
8 | def when_start():
9 |     com_num = random.randint(1, 50)
10 |
11 |     while True:
12 |         Entry.input("마음 속으로 1부터 50까지의 수를 생각했으니, 한번 맞춰봐! 몇 번만에 맞출 수 있을까?")
13 |         my_num = Entry.answer()
14 |         try_total += 1
15 |
16 |         if my_num == com_num:
17 |             Entry.print("Bingo! " + try_total + " 번 만에 찾아냈군!")
18 |             Entry.stop_code("all")
19 |         else:
20 |             if my_num < com_num:
21 |                 Entry.print(my_num + " 보단 더 높은 수야")
22 |             else:
23 |                 Entry.print(my_num + " 보단 더 낮은 수야")
24 |             Entry.wait_for_sec(2)
```

 9번 라인에서 무작위 수를 만드는 randint 함수를 사용하였다. 해당 함수는 random이라는 모듈(또는 라이브러리) 안에 존재하기 때문에 오리 지널 파이썬이라면 해당 모듈을 불러드려야 하기 때문에 import random 이란 코드가 코드 맨 상단에 추가되어야 함에도 불구하고, 엔트리-파이썬 에서는 이러한 import 하는 과정없이도 해당함수를 직접 호출하여 사용하고 있다. 함수에 넘겨질 값은 어떤 수부터 어떤 수까지의 사이의 무작위 수 인지를 정확히 요청하기 위해 1~50 사이의 숫자란 의미로, randint(1, 50)의 시작과 마지막 값, 두 값을 함수의 인자값(아규먼트)들로서 넘겼다.

 14번 라인에서 try_total += 1 이라는 구문에서 낫선 연산자 '+=' 가 등장했는데, 사실은 이 연산의 사용은 이전 장의 리스트 사용예제에서 총 합계를 구하는 식에서 이전 합계에 현재 값을 누적해 더해가는 코드와 동일한 코드이다. 따라서, 그 예제처럼 try_total = try_total + 1 이라고 적 어도 동일하게 동작하고, 이 코드를 간소화해서 try_total += 1 이렇게 적어도 동일한 의미가 된다. 즉, 이전의 try_total값에서 연속적으로 1씩 더해나가는 의미가 된다.

그밖에 나머지 코드들은 이미 배운 것으로 크게 이해에 어려운 것이 없을 것으로 예상되어, 추가적인 설명은 필요 없을 것 같고, 이제 이 게임을 더 재미있게 발전시켜 보는 것은 여러분의 과제로 남겨본다.

3.8 함수 (Function)

드디어 이제 마지막으로 이번 과의 마지막인 함수 파트만을 남겨두고 있다. 함수는 지금까지 이미 누가 우리를 위해 만들어 놓은 함수만을 호출해서 사용해 보았을 뿐, 나만의 내 함수(사용자 함수)를 직접 만들어 본적이 아직 없는데 이제 그 함수를 직접 만들어 보는 것을 해보도록 하겠다. [3.7 장](#)에서 만들었던 게임 코드의 일부를 함수로 별도 구현해 동일한 기능구현을 해보는 방식으로 함수를 배워보자.

먼저 코드 중에서 어떤 부분을 사용자 함수(사용자의 개인함수)로 따로 떼어내어 만들지를 정해야 하는데, 사용자가 입력한 수와 컴퓨터가 속의 생각한 수를 비교하는 부분을 떼어서 나만의 사용자 함수를 만들고 그쪽으로 두 수의 비교처리를 위임해 보도록 하자.

실행결과

kom_num29

my_num0

try_total0

대답0

마음 속으로 1부터 50까지의 수를 생각했으니, 한번 맞춰봐! 몇 번만에 맞출 수 있을까?



31

✓

kom_num29

my_num31

try_total1

대답31

31 보단 더 낮은 수야



kom_num29

my_num29

try_total3

대답29

Bingo! 3 번 만에 찾아냈군!



kom_num 29
my_num 29
try_total 3

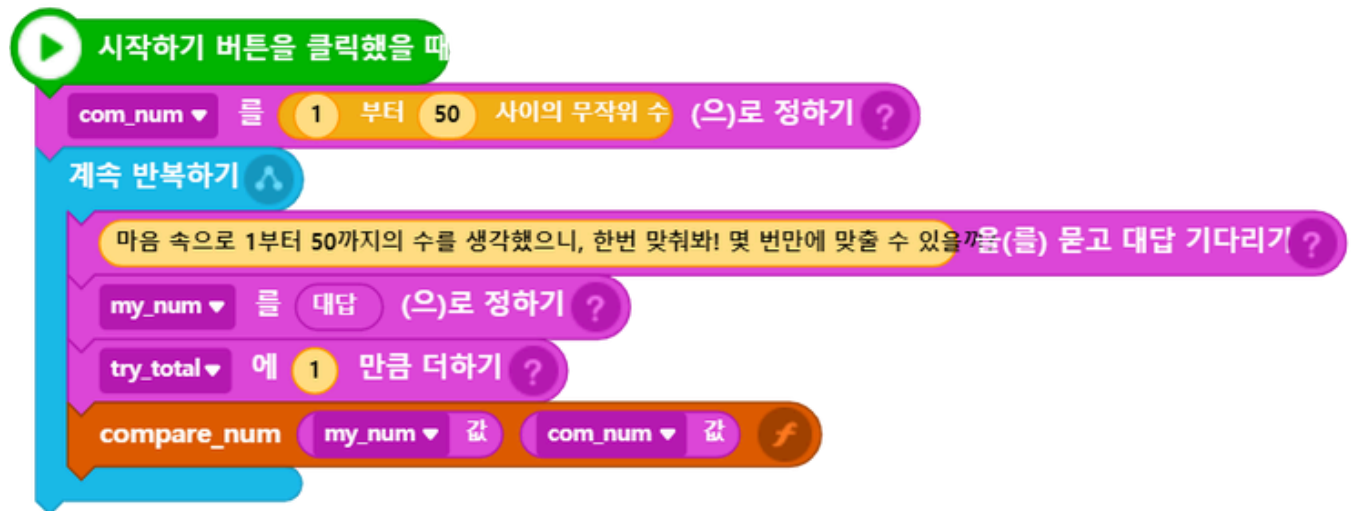
블록코딩(사용자 함수 미사용)



엔트리-파이썬(사용자 함수 미사용)

```
1 | # 엔트리봇 오브젝트의 파이썬 코드
2 | import Entry
3 |
4 | com_num = 0
5 | my_num = 0
6 | try_total = 0
7 |
8 | def when_start():
9 |     com_num = random.randint(1, 50)
10 |
11 |     while True:
12 |         Entry.input("마음 속으로 1부터 50까지의 수를 생각했으니, 한번 맞춰봐! 몇 번만에 맞출 수 있을까?")
13 |         my_num = Entry.answer()
14 |         try_total += 1
15 |
16 |         if my_num == com_num:
17 |             Entry.print("Bingo! " + try_total + " 번 만에 찾아냈군!")
18 |             Entry.stop_code("all")
19 |         else:
20 |             if my_num < com_num:
21 |                 Entry.print(my_num + " 보다 더 높은 수야")
22 |             else:
23 |                 Entry.print(my_num + " 보다 더 낮은 수야")
24 |         Entry.wait_for_sec(2)
```

블록코딩(사용자 함수로 구현)



엔트리-파이썬(사용자 함수로 구현)

```
1 | # 엔트리봇 오브젝트의 파이썬 코드
2 | import Entry
3 |
4 | com_num = 0
5 | my_num = 0
6 | try_total = 0
7 |
8 | def compare_num(num1, num2):
9 |     if num1 == num2:
10 |         Entry.print("Bingo! " + try_total + " 번 만에 찾아냈군!")
11 |         Entry.stop_code("all")
12 |     else:
13 |         if num1 < num2:
14 |             Entry.print(num1 + " 보단 더 높은 수야")
15 |         else:
16 |             Entry.print(num1 + " 보단 더 낮은 수야")
17 |         Entry.wait_for_sec(2)
18 |
19 | def when_start():
20 |     com_num = random.randint(1, 50)
21 |
22 |     while True:
23 |         Entry.input("마음 속으로 1부터 50까지의 수를 생각했으니, 한번 맞춰봐! 몇 번만에 맞출 수 있을까?")
24 |         my_num = Entry.answer()
25 |         try_total += 1
26 |
27 |         compare_num(my_num, com_num)
```

우리가 이 과의 맨 첫 장에서 함수를 어떻게 만드느지를 배웠기 때문에, 아직 함수 만드는 문법에 대한 이해가 부족하다면 해당 과로 되돌아 가서 복습하고 돌아올 것을 권한다.

8~17번 라인의 코드가 기존의 프로그램 코드의 16~24인을 떼어내어 `compare_num`이란 사용자 함수를 만든 부분이다. 함수의 파라미터로 2개의 숫자값을 전달 받아오게 되어 함수 내부에서 그 두 수의 크기비교 결과에 따른 조건적 실행결과를 보여주게 된다. 함수를 만들고 사용하는 법을 이미 숙지했다면 전혀 이해에 어려움이 없는 부분이다.

다만, 해당 사용자 함수는 사실상 효용성이 많이 떨어지는 사용자 함수로서 왜냐하면 함수의 주된 목적이 프로그램 코드 내에서 여러 부분에서 반복되어 되어 사용되는 기능을 하나의 함수로 대체하여 전체 코드의 간소화, 그에 따른 코드의 가독성 향상, 추후 발생할 수 있는 코드수정 대응에 용이

성 등 여러가지 이점을 누리기 위한 목적인데, 방금 만들어낸 사용자 함수의 경우는 그 목적에 크게 부합하지는 못하는 단순 학습목적의 사용자 함수로 나만의 함수를 직접 만들어 보고 사용해 보기 위한 인위적인 예시 목적이 더 크다고 할 수 있겠다.

그밖에 사용자 함수가 더 많은 곳에서 활용되기 위해서는 가능한한 보편적 기능을 구현할 수록 좋는데 예를들어, 지금처럼 크기비교+비교결과와 화면출력까지 묶어서 기능구현하는 것보다(이처럼 여러 기능이 하나로 묶일 경우, 특정 코드에서만 제한적으로 사용가능하게 되는 단점이 발생) 단순히 크기비교 기능만을 별도 개별 구현한 함수가 되는 것이 함수 활용성을 높일 더 좋은 구현이나 아쉽게도 현재 엔트리-파이썬 수준에서는 이런식의 파이썬 코드구현이 불가능한 한계(함수 결과값을 함수 호출자에게 회신(리턴)하는 기능의 미구현)가 있어서 불가피한 구현이 되었음을 염두해 두길 바란다.

그밖에 위와 같이 함수를 코딩한 후에 시작하기 버튼을 눌러 앱을 실행 후 다시 종료했을 때, 방금 전에 작성한 나의 함수코드가 갑자기 사라져버린 것처럼 보이는데, 실은 해당함수는 블록코딩 내의 함수 카테고리 안으로 자동등록되어 버리면서 코드가 그쪽으로 옮겨져 버린 것이다. 이 부분 역시도 아쉽게도 엔트리-파이썬에서 함수코딩시 사용자 입장에서는 혼동될 수 있는 부분이다.

4. 엔트리-파이썬으로 프로그래밍의 핵심 개념을 익히자

이번 장에서부터는 언어 그 자체의 문법이 아닌 프로그래밍의 패러다임이라 불리는 개념들에 대한 것들과 그 패러다임 안에서의 핵심요소들에 대한 것들을 엔트리-파이썬에서 약간이나마 맛보기 하는 것으로 엔트리-파이썬의 학습 전체를 마무리지면 좋을 것 같다.

패러다임이라는 것은 일반용어상 사전적 의미로 사고(思考)를 구성하는 인식의 체계라 할 수 있는데 프로그래밍 세계에서도 유사한데 프로그래밍을 할 때 어떤 사고적 인식의 틀 안에 일관된 경향성을 갖고, 그 틀 안에서 코딩을 해나가는 것이다. 이런 식의 의미가 잘 와닿지 않을 것 같은데, 계속 앞전에서 텍스트코딩을 한 외국어를 독파하는 것으로 빗대어 설명한 것에 연장선에서 먼저는 외국어의 문법체계를 익혔다면, 패러다임은 기본적인 문법체계는 다 안다는 것에 기반해, 이제는 어떤 형식을 갖는 글을 써나갈 것인가의 것으로 빗대면 이해가 더 쉬울 것 같다. 예를 들어 글쓰기에도 장르가 있는데, 일반 수필과 영국/영화 시나리오의 글쓰기는 엄연히 형식과 내용에서 크게 다른 것처럼 코딩을 할 때도 목표한 최종 프로그램에 적합한 특정 패러다임을 사전에 정하고 그러한 형식과 틀 안에서 코딩을 한다면 생각하면 좋을 것 같다. 그런데 글쓰기에서도 장르파괴(?)라고 해서 여러 장르가 혼합되는 것이 가능하듯이, 실제 텍스트코딩에서도 여러 패러다임이 섞여 코딩되는 것도 가능하다는 점도 알아두면 좋겠다.

사실은 이러한 패러다임의 개념들이 이미 엔트리 블록코딩 안에 다 녹아있는데, 우리는 그동안 인지하지 못한 채 코딩을 해왔던 것이다. 나는 텍스트코딩까진 관심없고 블록코딩에서 내 코딩의 끝을 맞이하겠다 한다면 사실 끝까지 모르고 지나가도 될 수 있고, 모르고도 블록코딩으로 끝까지 재미있게 놀 수 있다. 하지만, 우리는 이제 블록코딩의 세계(world)에서 다리를 건너 텍스트코딩 세계로 건너가고 있으며, 텍스트코딩 세계에서 코딩한다는 것은 그 세계에서 살아가야 할 또다른 차원의 지식의 필요하고, 이를 알고 코딩하는 것과 모르고 하는 것은 텍스트코딩 세계에서 더 수월한 삶(?)을 살아갈 기반이 되며, 텍스트코딩 세계에서는 다른 동료들과 협력하는게 더 일상인데 그들과 코웍(co-work) 함에 있어서도 기본이 되는 배경지식을 쌓는 과정으로 여기면 될 것 같다.

4.1 순차/병렬처리 (Serial/Parallel)

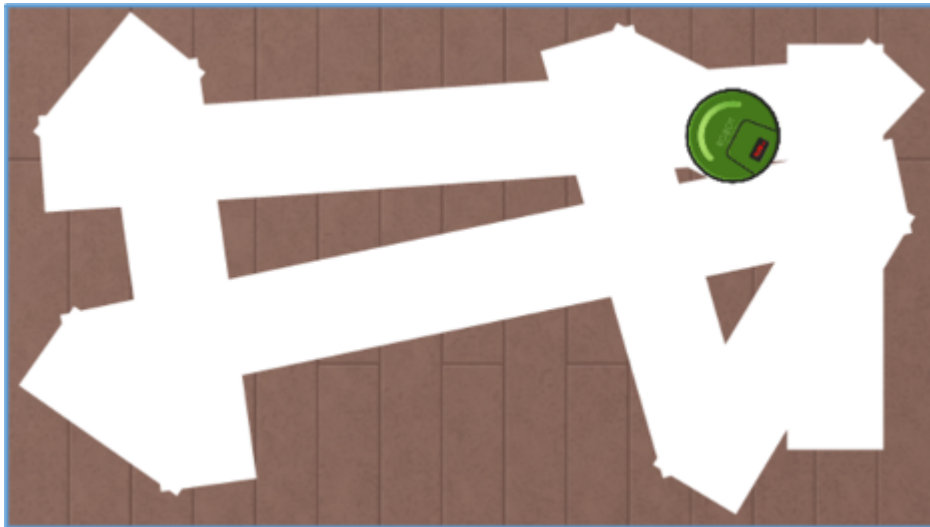
순차/병렬처리에 대한 개념은 사실은 프로그래밍의 패러다임까지는 아니다. 간단히 말하면 어떠한 일을 처리하는 순서로 복수업무를 순차적으로 처리할 것인가, 동시에 처리할 것인가 정도로 구분하면 되겠다.

순차처리(Serial processing)는 코드 한줄 한줄 순차적으로 수행된다는 것으로 언플러그드(unplugged) 교딩교육에서도 가장 기본으로 다루는 개념으로 사실상 우리가 너무 익히 잘 아는 개념이라 따로 설명이 필요없고, 병렬처리(Parallel processing)라는 컴퓨터공학의 원론적인 개념으로 설명하기 시작하면 초보자 수준에서 이해하기 어렵고, 또 엔트리-파이썬에서 그러한 수준에 병렬처리를 지원하지도 않아서, 간단히 개념적으로만 이해하고 넘어가도 좋겠다. 병렬처리는 복수의 둘 이상의 작업을 동시에 실행할 수 있는 방식이라고 말했다. 이때 진짜 각각의 작업들이 완전히 독립적으로 개별로 실행되는거나(병렬성, Parallelism), 마치 작업들이 동시에 실행되는 것처럼 보이게 하는 테크닉(동시성, Concurrency)으로 실행되게 하는 것이냐에 따라 구분이 많이 달라지는데, 엔트리-파이썬은 후자라고 하겠다.

엔트리-파이썬에서는 동시성(concurrency) 처리 프로그래밍시 사용하는 일반적인 기법(쓰레드, 멀티 프로세스 등)을 사용해 코딩할 수조차 없으니 당연히 동시성 텍스트코딩하는 방법도 배울 수가 없다. 이 책의 서두에서도 언급했지만, 엔트리-파이썬의 갖는 한계임과 동시에 애시당초 엔트리-파이썬에서 그정도까지의 학습을 의도하지 않았고, 블록코딩에서 첫 텍스트코딩의 넘어가는 분들이 대상이라 개념적 이해만 하고 넘어가도 충분하고, 사실상 그것만으로 가치는 분명히 있다는 것을 알면 좋겠다.

엔트리 블록코딩에서 다음의 예제와 같이 우리는 이미 병렬처리를 많이 코딩해봤다.

실행결과



블록코딩



엔트리-파이썬

```
1  # 로봇청소기(1) 오브젝트의 파이썬 코드
2
3  import Entry
4
5  def when_start():
6      Entry.start_drawing()
7      Entry.set_brush_color_to("#FFFFFF")
8      Entry.set_brush_size(50)
9
10 def when_start():
11     while True:
12         Entry.move_to_direction(10)
13         if Entry.is_touched("edge"):
14             Entry.move_to_direction(-15)
15             Entry.add_rotation(133)
```

"시작하기 버튼을 클릭했을 때" 라는 블록을 한 오브젝트 내에서도 여러 개 사용해 독립적으로 보이는 작업들을 마치 동시에 처리하듯이 코딩해본 경험이 있는 것이다. 이처럼 우리는 여러가지 프로그래밍의 핵심개념들을 알게 모르게 사용해왔던 것이다.

따라서, 엔트리-파이썬에서의 병렬처리는 블록코딩에서 해본 것과 크게 다르지 않다. 단지, 위의 예저처럼 when_start 함수를 두번 이상 사용할 수 있어, 사용자가 프로그램을 시작하기 위해 시작하기 버튼을 누름과 동시에 각각 when_start 함수가 동시에 불려지기 때문에 그 함수 호출시 수행해야 할 여러작업을 독립적으로 기술하고, 마치 동시에 실행되는 것 같은 효과를 누리도록 코딩하고 활용할 수 있으면 되겠다.

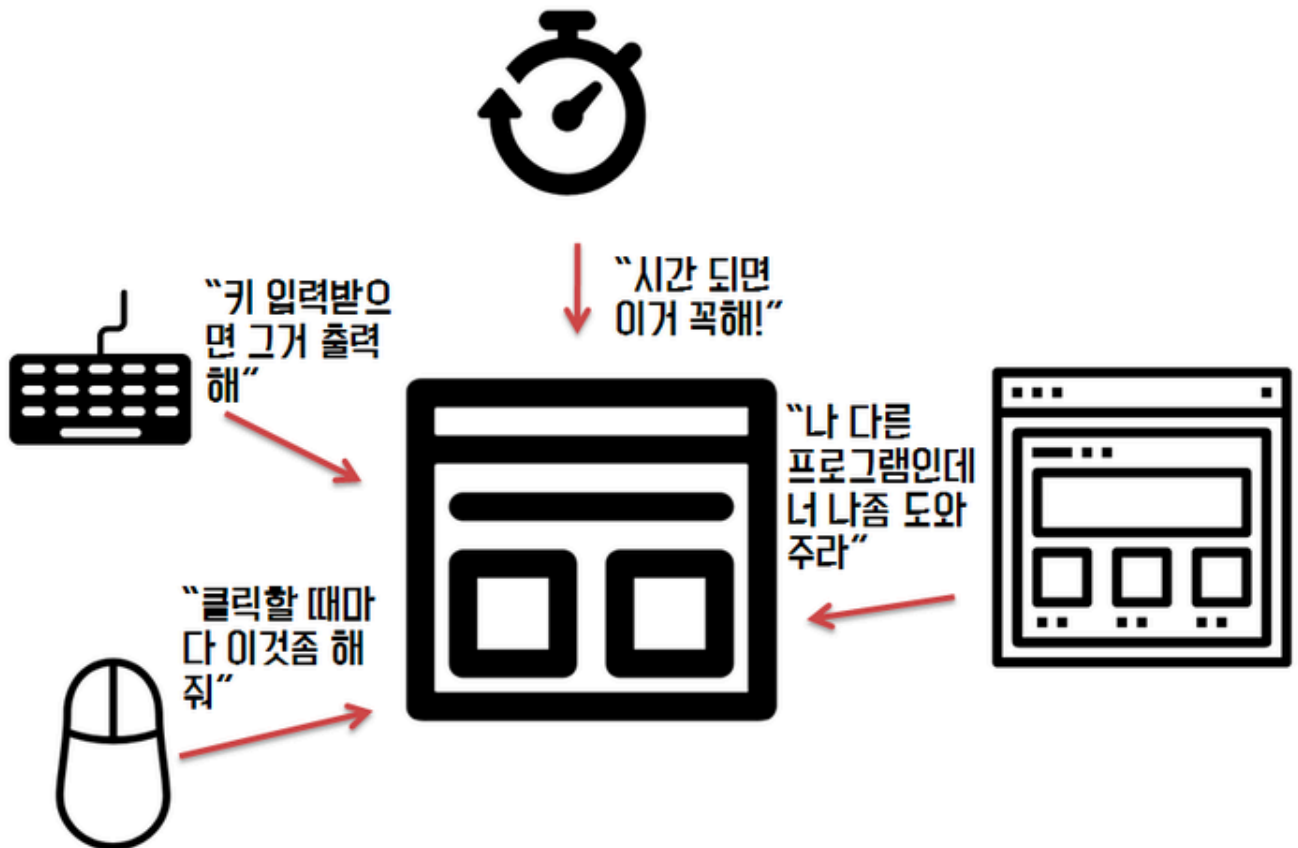
4.2 절차 지향형 (Procedural)

프로그래밍 패러다임 중에 절차 지향형 이라는 패러다임이 있다. 아주 전통적인 프로그래밍에서 가장 먼저 등장했던 패러다임이고, 우리가 알게 모르게 엔트리에서 블록코딩에서도 이미 사용해 왔던 패러다임 중에 하나이다. [3.8장](#)에서 함수에 필요성과 목적에 대해 잘 이해했듯이 프로그램 안에서 재사용성(Reusability)이 큰 코드덩어리를 함수(Function)로 분리하고 그 함수 호출해 활용해서 코딩해 나가는 방식을 절차 지향형 패러다임이라고 말한다. 과거에 함수는 함수(Function)라는 용어보다 프로서저(Procedure) 또는 서브루틴(Sub-routine) 등으로 불리었는데, 그래서, 절차지향형이라는 말의 영어표현이 Procedural라고 불리고 있다.

절차지향의 예제는 이미 이전 장에 함수에서 배운 예제와 별다를게 없으므로 해당 예제로 갈음하는 것으로 하겠다.

4.3 이벤트 주도형 (Event-Driven)

이벤트 주도형 또는 이벤트 기반형(Event-Based) 프로그래밍 패러다임에 대해 알아보자. 이 패러다임이 대두되기 시작한 것은 사실한 운영체제(OS: Operating System)이 기존에 콘솔(Console)이라 불리는 텍스트기반으로 명령어를 일일이 입력해 운영체제와 소통하던 인터페이스에서 GUI(Graphic User Interface, 그래픽 사용자 인터페이스)방식으로 전환되기 시작하면서 대두된 것으로, 즉, GUI 환경 맞는 형식에 프로그램이 필요했고, 프로그래밍 개발방식도 마찬가지로 이에 맞춘 대응이 필요했다고 볼 수 있겠다. GUI 환경이란 다르게 아니고, 그래픽 화면 기반에서 시스템(즉 컴퓨터)를 다루는 것으로 PC든 스마트폰이든 클릭과 터치로 선택하고 조작하고 하는 것이 이미 우리가 컴퓨터를 다루는 보편화된 환경이 되어있기 때문에 이 프로그래밍 패러다임의 이해에는 크게 어려움은 없을 것 같다. 그럼에도 이벤트 주도형의 개념을 다음의 그림을 보고 더 잘 이해해 보자.



출처: [Kamang's IT Blog](#)

지금 화면에 보이는 내 프로그램에게서 어떤 행동을 촉발(트리거: **Trigger**)되게 만드는 모든 상황이 사건(이벤트)라 지칭할 수 있다. 사용자가 화면 상에 특정 이미지 버튼(또는 아이콘)을 누른 시점, 특정 키보드 자판의 키(알파벳, 특수문자, 방향키 등)를 누른 시점, 마우스의 특정 버튼(왼쪽/중간/오른쪽 등)을 누른 시점, 어떤 예정된 시간이 된 시점, 다른 프로그램 자체로부터의 내 프로그램을 호출하는 시점 등등 말이다.

즉 개발자는 이러한 이벤트가 발생할 것을 미리 사전에 예상하고, 이러한 각각의 이벤트가 발생했을 때 어떤 일을 수행하라 라는 식으로 개념을 갖고 코드작성을 해나가는 식의 프로그래밍의 방식을 이벤트 기반형 프로그래밍 패러다임이라 할 수 있다. 우리는 이미 엔트리에 이러한 방식으로 코드작성을 해본 경험이 많아서 아마도 개념적 이해나 코드적용에는 크게 어려움이 없을 것으로 예상된다. 다만, 우리는 그것을 엔트리-파이썬에서는 어떤 함수를 활용해 코딩할 수 있는지만 알아보면 되겠다.

다음의 표에서 엔트리 안에 이미 존재하는 이벤트의 종류와 그 이벤트에 맞는 미리 사전 정의된(약속된) 콜백함수(3.1장 콜백함수 참조)의 목록을 확인할 수 있다.

이벤트 종류	블록코딩용 블록	텍스트코딩용 콜백함수
화면에 시작하기 버튼을 클릭했을 때	 시작하기 버튼을 클릭했을 때	def when_start():
키보드에 키를 눌렀을 때	 q ▼ 키를 눌렀을 때	def when_press_key("q"):
마우스 클릭했을 때	 마우스를 클릭했을 때	def when_click_mouse_on():
마우스 클릭을 해제했을 때	 마우스 클릭을 해제했을 때	def when_click_mouse_off():
오브젝트 클릭했을 때	 오브젝트를 클릭했을 때	def when_click_object_on():
오브젝트 클릭을 해제했을 때	 오브젝트 클릭을 해제했을 때	def when_click_object_off():
새로운 장면이 시작되었을 때	 장면이 시작되었을 때	def when_start_scene():

4.4 객체 지향형 (Object-Oriented)

마지막으로 객체지향형(Object-Oriented) 프로그래밍 패러다임에 대해 알아보자. 이 패러다임의 등장 이전과 이후로 나뉘만큼 소프트웨어 개발에 가장 큰 영향을 끼친 혁신적 패러다임으로 간주되고 있으며, 특히나 프로그램의 크기가 크거나 여러 개발자들과 협업개발이 필요할 때, 보편적으로 활용되어지고 있으므로 잘 이해하고 알아두는 것이 필요하다. 하지만, 이 개념을 정확히 이해하고 자유자재로 활용하기에는 깊은 이해가 필요한 게 사실이고, 깊이 들어가면 들어 갈수록 녹록지 않은 면도 실제 있으니, 그런 깊은 이해까지는 추후에 필요한 시점에 더 찾아서 공부하면 될 일이기 때문에, 엔트리-파이썬에서는 기본적인 개념이해만 확실히 해두어도, 텍스트코딩 시작에 있어서는 큰 자산이 될 수 있다.

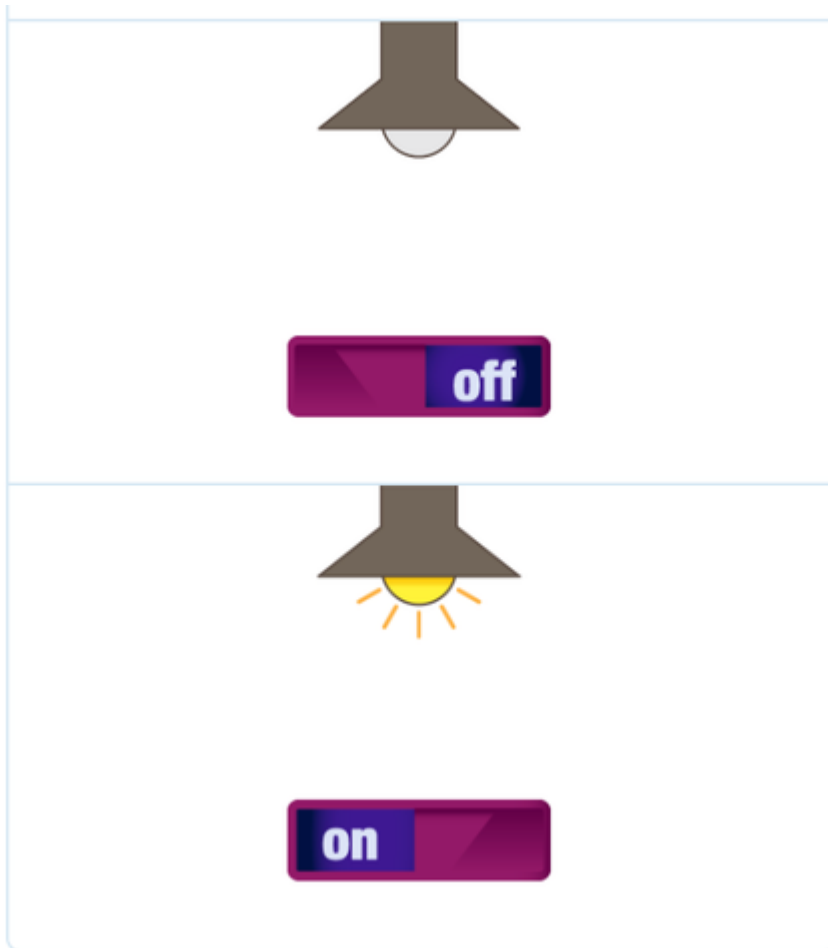
이전 장들에서 언급된 패러다임들도 그게 무엇인지 모르게 자연스럽게 익혀온 것처럼 엔트리가 처음 만들어지면서부터 의도했는지 모르겠으나, 우리가 이 패러다임을 어렵지 않게 받아드리도록 잘 계산된 설계가 있었던 생각이 든다. 먼저, 우리가 엔트리에서 코딩을 하기 전 쯤 먼저 해야할 일이 무엇이었는지 상기해보자. 그렇다 바로 오브젝트를 최소한 1개 이상을 화면 추가하는 일이었다. 여기서 오브젝트(Object)라는 용어를 도입한 것부터 사실상 이 개념을 이해시키기 위한 잘짜여진 각본이 있었던 생각이든다.

엔트리에서는 실행화면에 등장하는 모든게 오브젝트이다. *그냥 이미지냐 오브젝트냐 아니냐의 구분은 해당 개체에 독립된 코드를 붙혀 조작할수 있는지의 여부에 달려있다.* 우리는 엔트리 블록코딩을 시작할 때 각양각색의 오브젝트들(각종 이미지형 오브젝트들(배경 오브젝트 포함), 글상자라고 불리는 텍스트형 오브젝트 등)을 가져다가 화면에 배치하는 일부터 시작했었다.

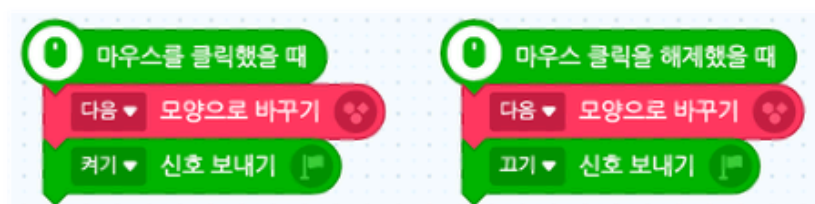
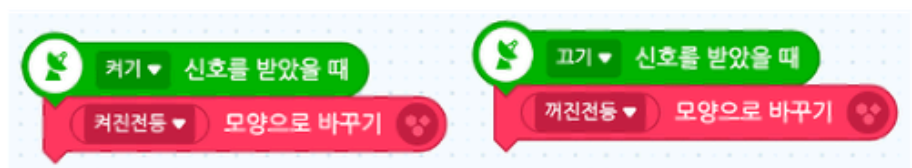
그리고, 그 배치된 각각의 오브젝트들에게 생명력을 주기 위해 각 오브젝트에게 역할과 활동에 적합한 해당 오브젝트 만의 독립된 코드를 작성하게 되고, 때로는 각 오브젝트들이 서로 협업을 할 필요가 있게되면, 오브젝트들 사이에 어떤 요청을 하는 메시지를 주고 받으면서, 상호 협업하는 구조를 만들면서 전체 프로그램을 만들어가는 식의 코딩을 해왔다. 지금까지 언급된 이러한 일련의 과정이 사실상 바로 객체 지향형 패러다임을 사용한 개발과정이었고, 텍스트코딩 세계에서는 이를 객체 지향형 프로그래밍(OOP: Object Oriented Programming) 이라고 하여, 마치 고유명사처럼 부르고 있다.

자 이제 예제를 통해 오브젝트들이 메시지를 상호협력을 위해 메시지를 주고받는 코드를 확인해보자.

실행화면



블록코딩



텍스트코딩



```
1 # 전등 오브젝트의 파이썬 코드
2
3 import Entry
4
5 def when_get_signal("켜기"):
6     Entry.change_shape("켜진전등")
7
8 def when_get_signal("끄기"):
9     Entry.change_shape("꺼진전등")
```



```
1 # 스위치(1) 오브젝트의 파이썬 코드
2
3 import Entry
4
5 def when_click_mouse_on():
6     Entry.change_shape_to("next")
7     Entry.send_signal("켜기")
8
9 def when_click_mouse_off():
10    Entry.change_shape_to("next")
11    Entry.send_signal("끄기")
```

사실 실제 텍스트코딩에 기반한 객체지향에서의 협업의 과정은 위와같은 중간매개가 존재하는 메시지전달 방식보다 각각 객체 내에 존재하는 외부 인터페이스 함수들을 외부의 다른 객체에서 직접호출을 통해 이뤄지는게 일반적이다. 지금 이러한 설명은 객체 지향형 패러다임의 깊은 이해를 요하는 내용이라, 이러한 내용을 여기서 자세히 설명하는 것은 본 서의 범위를 넘어서는 것이라 만약, 더 관심이 가는 분들이 계시다면, 객체 지향형 패러다임에 대한 전문서와 함께 특별히 파이썬에서 객체 지향형 패러다임을 어떻게 구현하는지에 대해 꼭 공부해 보기를 추천한다.

에필로그

이로서 본 서의 마지막인 이곳까지 달려오신 여러분의 수고에 박수쳐 드리고 싶고, 텍스트코딩 세계(world)에 본격 입문하신 것도 다시한번 축하 드린다. 저자는 여러분의 텍스트코딩 세계 안에서의 재미있는 여정을 돕기 위해 계속해서 [그 다음 레벨의 책](#)을 저술 중에 있다. 그 책도 세상에서 많은 이들의 생각과 사고의 지평을 넓히고 자유롭게 되는데 사용되고 기여할 수 있길 바라고 있다. 2023년 가을의 끝자락에서... 저자올림

5. 부록

코딩에 도움이 되는 단축키 목록 등을 안내합니다. 일반적으로 개발자들은 빠른 작업을 위해 단축키 활용에 능숙합니다. 여러분들도 단축키를 활용하는 습관 들이기를 추천합니다.

엔트리-파이썬 단축키 모음

윈도우즈용

기능	단축키
들여쓰기 / 내어쓰기 단계 조정	tab , shift + tab
블록 / 모양 / 소리 / 속성 탭 선택	Alt + 1 , Alt + 2 , Alt + 3 , Alt + 4
이전 / 다음 오브젝트 선택	Alt + [, Alt +]
블록 코딩 / 엔트리파이선 모드 변경	Ctrl + [, Ctrl +]
실행	Ctrl + R

맥용

기능	단축키
들여쓰기 / 내어쓰기 단계 조정	cmd + [, cmd +] tab , shift + tab
블록 / 모양 / 소리 / 속성 탭 선택	Alt + 1 , Alt + 2 , Alt + 3 , Alt + 4
이전 / 다음 오브젝트 선택	Alt + [, Alt +]
블록 코딩 / 엔트리파이선 모드 변경	Ctrl + [, Ctrl +]
실행	Ctrl + R